



CMPG311

PHASE 1:

DATABASE INITIAL STUDY

GROUP 18: 404 NOT FOUND

Delphy Stone (Group leader): 50509772

Mbali Mlambo: 50763458

Tholoana Mothibi: 50371649

Itumeleng Maleka: 54122104

Khuthadzo Mudau: 51978156

Submission date: 2026/04/17

1. Table of Contents

Members of the group	3
1.Background:	5
2.Company situation:.....	5
Company objectives	5
Company operations	6
Company organizational structure	7
3. Problems and constraints:.....	8
Problems	8
Constraints.....	8
4.Database system specification:.....	9
Objectives to solve identified problems.....	9
Information required by company from database.....	10
Scope.....	11
Boundaries	11
Conclusion:	12

2. Members of the group

MEET THE TEAM

	Name & Student number	Contact details
	Delphy Stone (Group leader) 50509772	50509772@mynwu.ac.za
	Khuthadzo Mudau 51978156	51978156@mynwu.ac.za

	Name & Student number	Contact details
	Mbali Mlambo 50763458	50763458@mynwu.ac.za
	Tholoana Mothibi 50371649	50371649@mynwu.ac.za
	Itumeleng Tumelo Maleka 54122104	54122104@mynwu.ac.za

3. **1.Background:**

SkyLink Airline is a commercial airline that offers only economy-class flights. It was established in 2021 and is located in the North-West province of South Africa. This airline offers domestic air transport services to passengers traveling between provinces within South Africa. Its main aim is to offer safe, affordable, efficient and reliable air travel services. As the airline grew, the volume of operational data increased significantly. This data includes passenger details, flight schedules, bookings, aircraft records and staff details. Initially, the company relied on an outdated database which caused several operational problems. For instance, the system occasionally double-booked seats, resulting in multiple passengers with the same seat. These challenges affected the efficiency of the airline negatively. As the airline continues to grow, the implementation of a well-designed database system is crucial to ensure improved operational management, accurate data storage and efficient information retrieval.

4. **2.Company situation:**

Company objectives

1. Provide safe and reliable air transportation:

Ensure passenger safety and arrive on time by effective management of aircraft, flight and passenger information.

2. Improve booking and ticket management:

Provide an efficient system to book and manage tickets while preventing duplicate seat bookings.

3. Ensure accurate payments processing:

Keep a record of payments of tickets to confirm bookings.

4. Improve flight and aircraft management:

Keep a record of the flights and aircrafts to properly support the allocation and scheduling.

5. Improve customer service:

Provide staff with quick access to passenger and booking information to improve the overall customer experience.

6. Support route expansion:

Expand the business by adding more flight routes to additional provinces and reach more customers and become more competitive.

7. Increase revenue and profitability:

Improve efficiency in bookings, payments, and flight management to attract more customers and becoming more competitive.

8. Support better reporting and decision making:

Generate reports to obtain information about passengers' numbers, bookings and flight schedules to assist in decision-making.

9. Promote operational efficiency:

Improve management of resources and staff information to ensure the airline operates efficiently.

Company operations

SkyLink Airline carries out several operational activities in order to ensure efficient transportation of passengers. The main operations of SkyLink include:

- 1. Flight scheduling:** Ensure efficiency by planning flight routes, departure times and arrival times for all flights.
- 2. Aircraft management:** The airline must maintain the records of all aircrafts, including operational status and seating capacity.
- 3. Passenger bookings:** Passengers must be able to book available seats on a scheduled flight after paying for the ticket and providing the required identification documents. The system should be accessible 24 hours a day to allow bookings at any time.
- 4. Seat allocation:** Each passenger should be assigned a specific seat on a flight, and no seats should be assigned to more than one passenger.
- 5. Crew management:** Pilots and flight attendants must be assigned to one flight at a time according to their availability and ensure that the crew is not assigned to multiple flights at the same time.
- 6. Luggage management:** The airline will manage checked in luggage for the flights. The system will associate each luggage to a passenger to ensure proper tracking and handling.

Company organizational structure

1. Reservations and booking department

- **Reservations Agent:** Responsible for updating passenger bookings in the system when passengers change flights. The agent also manages the process of flight cancellations.
- **Ticketing Officer:** Assists passengers with booking enquiries and is responsible for generating and issuing physical tickets for passengers.

2. Airport Operations(Check-in staff)

- **Check-in Agent:** Verifies passenger identification and booking details at the airport during the check-in process. The agent also checks the luggage weight of each passenger to ensure it does not exceed the permitted weight limit and assigns seats to passengers before boarding.
- **Boarding Gate Agent:** Confirms passenger boarding, verifies tickets and boarding passes, and updates passenger boarding status in the system.

3. Finance Department

- **Finance Manager:** Oversees budgeting and financial planning for the airline. The manager also monitors income generated from bookings and reviews financial reports of the business.
- **Accounts Officer:** Processes passenger payments and manages refunds when required and maintains accurate financial transaction records in the business.

4. Flights Operations Department

- **Flight Scheduler:** Responsible for creating and updating flight schedules and assigning aircraft to specific flights.
- **Flight Operations Manager:** Coordinates aircraft and crew assignments and ensures that flights operate according to schedule.
- **Flight Engineer:** Monitors aircraft performance, performs safety checks and reports technical issues related to aircraft systems.
- **Pilot and Co-pilot:** Responsible for navigating and operating the aircraft during a flight. They communicate with air traffic control during take-off, flight and landing to ensure safe operation of the aircraft.
- **Flight Attendant:** Assist passengers during boarding and seating, demonstrate safety procedures before take-off and provide in-flight services such as food and refreshments.

5. Administration

- **Operations Manager:** Monitors airline operations and overall performance and reviews flight reports to ensure efficient operations.
- **Systems Administrator:** Manages the airline database system, ensures system security and controls user access to the database.

5. 3. Problems and constraints:

Problems

1. **Double booking of crew members:** Crew members, such as pilots or flight attendants may be assigned to more than a single flight at a given time. This can cause operational issues such as flight delays or cancellations due to the unavailability of crew members.
2. **Data integrity issues:** The system might have stored incorrect or incomplete data. For instance, the system might have stored a passenger's data without an ID/passport number or stored booking information without a flight number.
3. **Data inconsistency:** The system data may look different in various parts of the system due to a lack of synchronized updating. For instance, a flight time or a passenger's seat may be changed, and that change may not be reflected in the entire system.
4. **Limited storage capacity for historical records:** The previous database system did not support large amounts of historical data, such as previous flights. As a result, some data was deleted or lost.
5. **Slow data retrieval:** It took a long time to search for bookings or flights due to inefficient database performance.
6. **Scalability limitations:** As the airline continues to grow, the existing system is unable to handle large quantities of data, resulting in slow performance.

Constraints

1. **Technical constraint:** The system must be developed using Oracle, limiting implementation to technologies compatible with Oracle.
2. **Data retrieval limitations:** Recovering historical data may be difficult due to limitations in the system, only allowing the retrieval of recently stored data.
3. **Budget constraint:** The system must be implemented within a given budget of R325 543.
4. **Aircraft weight:** Seeing that planes have a weight limit, passengers and luggage are included in the maximum weight limit of the aircraft, which is approximately 79 000 kg. The aircraft weight limit should not be exceeded. If the weight limit is exceeded that could cause safety risks.
5. **Time constraint:** The system must be completed before May 2026.
6. **Training constraint:** Employees may need to be trained properly to use the new system. However, the system training period is three weeks before system goes live, which may not be enough for the employee to fully understand the new system.

6. 4.Database system specification:

Objectives to solve identified problems

For SkyLink Airlines, the new operational database system aims to modernise and improve the airline's data management by replacing manual or non-efficient record keeping with a centralized digital system. By organizing key operational data in a structured relational database, the system will support accurate record keeping, smoother daily airline operations, and faster decision making. Specific objectives include:

1. **Staff information management:** The database will store and maintain accurate records of staff members, including their roles and responsibilities. This will allow for easy access and updates when necessary.
2. **Aircraft Management:** The database will maintain detailed records of aircraft used by the airline, ensuring accurate tracking of aircraft information and availability for flights.
3. **Airport Management:** The database will store and manage information about airports used in airline routes, supporting the organization of destinations and flight operations.
4. **Flight Scheduling:** The database will help manage and maintain flight schedules, ensuring that flights are well organized with correct routes, times and aircraft.
5. **Booking Management:** The database will help the airline to keep track and manage passenger bookings, ensuring that reservations are clearly and accurately recorded.
6. **Ticket Management:** The database will allow staff members to generate tickets; therefore, the database will keep track of ticket records ensuring each booking is properly documented.
7. **Operational Efficiency:** By centralizing airline data, the system will allow staff to quickly retrieve, update and manage information. This will then improve the workflow efficiency and reduce administrative delays.
8. **Luggage management:** The database will record and track passengers' luggage information to ensure that all bags checked in by the passengers are counted correctly, it will also reduce the risk of losing luggage.
9. **Flight weight management:** Database will help calculate and monitor total weight of passengers and luggage of each flight; this will ensure that the aircraft does not exceed its maximum weight capacity.

Information required by company from database

1. Staff Details:

The database will store information about staff members, including crew members and administrative personnel. Each record will include staff identification details, job roles, and contact information. This information allows staff members to be assigned to specific flights where required.

2. Aircraft Details:

The database will contain records of aircraft used by the airline. These records include details such as the aircraft identification number, seating capacity, and maintenance status. The maintenance status indicates whether an aircraft is available for flight assignment.

3. Airport Details:

The database will store information about airports within South Africa. Each airport record includes airport identification details and location information used to determine flight origins.

4. Booking Details:

When a passenger successfully books a flight, the booking information is stored in the bookings table. This information includes passenger contact details, selected flights, and booking status. These records are used to generate passenger tickets and produce reports identifying peak booking periods.

5. Flight Details:

The database will maintain records of flights, including departure times, destinations, and aircraft assignments. This information can be used to generate operational reports indicating the most frequently travelled destinations.

6. Passenger details:

During the booking process, passengers are required to indicate whether they meet the minimum age requirement to travel alone. Children under the age of 16 are not allowed to travel alone and must be accompanied by an adult. This information is recorded in the system to ensure that airline travel policies are followed.

Scope

The following functional requirements must be included:

- Maintain aircrafts
- Maintain airports
- Maintain aircraft routes
- Maintain bookings
- Maintain flights
- Maintain staff
- Maintain luggage
- Scheduling flights
- Make a booking
- Generate tickets
- Update flight departure status
- Most popular destinations report
- Monthly booking statistics report
- Aircraft route report
- Passenger maximum luggage weight report

Boundaries

1. **Promotions and marketing:** The system will not be able to handle marketing campaigns, advertisements, or promotions. This includes activities such as creating airline promotions, advertisements for flights, or handling marketing strategies.
2. **Live flight tracking:** The system will not be able to track the flight. Tracking the position of the aircraft during flight is usually done by specialized systems.
3. **Special meal options:** The system does not provide support for special meals such as kids meals, vegetarian meals, etc.
4. **Staff payroll management:** The system does not provide support for salary payment, payroll, etc. That will be handled by Human Resources.
5. **Government aviation certification and regulations:** The system will not be used to control pilot licenses, aviation regulations, or aviation authorities.
6. **Airport infrastructure management:** The system will not be used to control airport infrastructure such as runways, terminals, or airport security.

7. Conclusion:

In conclusion, SkyLink airline's need for a well-structured operational database is imperative to manage its growing operations. The current system has several challenges which affect service delivery and efficiency negatively. The proposed database will address this by supporting airline activities such as crew information management, aircraft management, flight scheduling and booking management.

This proposed system will improve data accuracy and reduce operational errors which will enable better decision-making and ultimately give customers a more satisfactory experience by ensuring system reliability.



CMPG311

PHASE 2:

DATABASE DESIGN

GROUP 18: 404 NOT FOUND

Delphy Stone (Group leader): 50509772
Mbali Mlambo: 50763458
Tholoana Mothibi: 50371649
Itumeleng Maleka: 54122104
Khuthadzo Mudau: 51978156

Submission date: 2026/04/17

8. Table of Contents

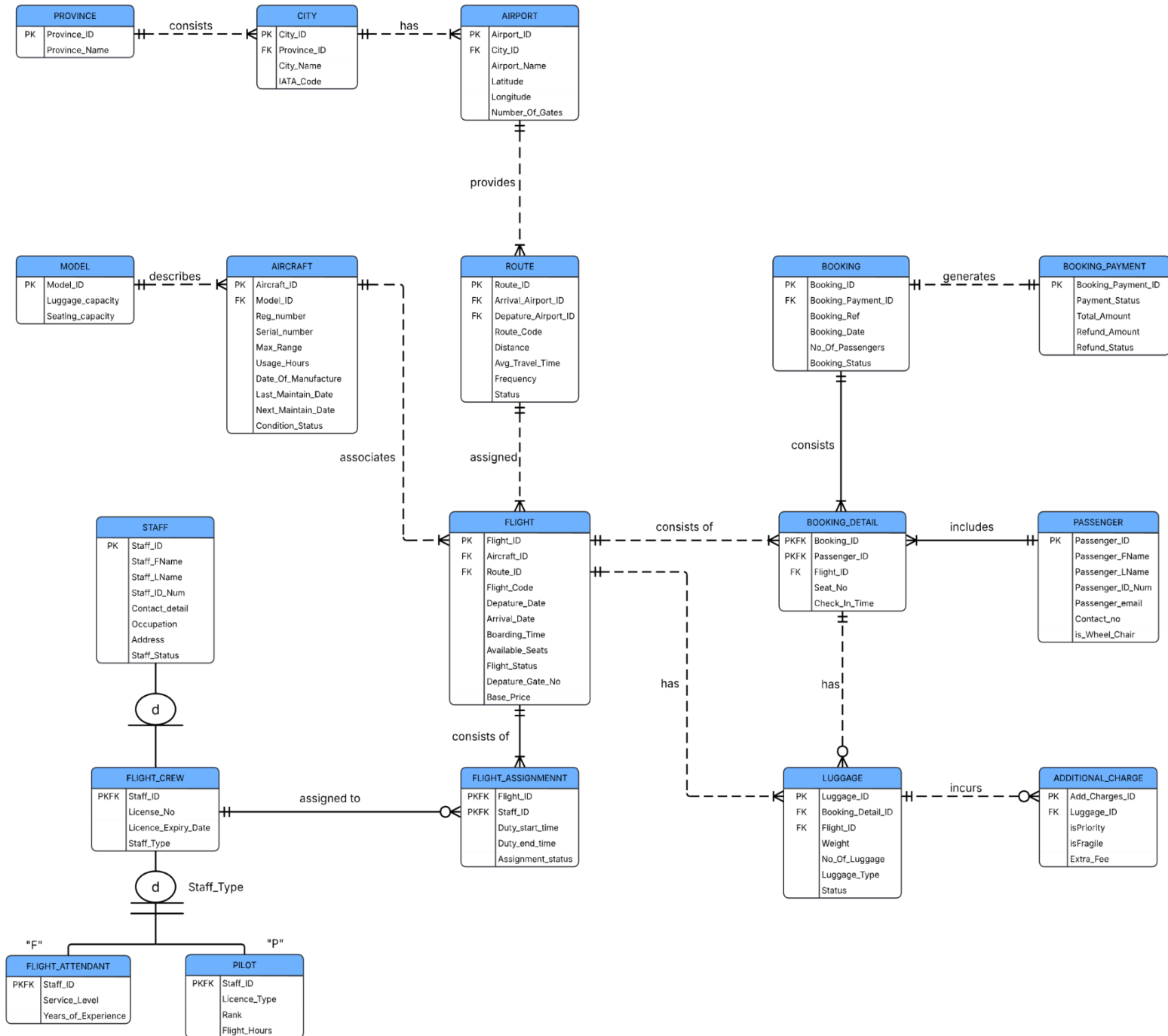
Business rules:	15
ER Diagram:	17
Notes:	18
Logical Design:	20
Dependency diagrams:	22

9. **Business rules:**

- **PROVINCE and CITY**
One PROVINCE can have one or many CITY(s)
Each CITY must belong to one PROVINCE
- **CITY and AIRPORT**
One CITY can have one or many AIRPORT(s)
Each AIRPORT must belong to one CITY
- **AIRPORT and ROUTE**
One AIRPORT can be associated with one or many ROUTE(S)
Each ROUTE has one departure AIRPORT
Each ROUTE has one arrival AIRPORT
- **ROUTE and FLIGHT**
One ROUTE can have one or many FLIGHT(s)
Each FLIGHT must use one ROUTE
- **AIRCRAFT and FLIGHT**
One AIRCRAFT can be used for one or many FLIGHT(s)
Each FLIGHT uses exactly one AIRCRAFT
- **MODEL and AIRCRAFT**
One MODEL can describe one or many AIRCRAFT(s)
Each AIRCRAFT belongs to one MODEL
- **FLIGHT and FLIGHT_ASSIGNMENT**
One FLIGHT consists of one or many FLIGHT_ASSIGNMENT(s)
Each FLIGHT_ASSIGNMENT is to be specific to one FLIGHT
- **FLIGHT_CREW and FLIGHT_ASSIGNMENT**
One FLIGHT_CREW member can be assigned to zero or many FLIGHT_ASSIGNMENT(s)
Each FLIGHT_ASSIGNMENT is specific to a FLIGHT_CREW member.
- **FLIGHT and LUGGAGE**
One FLIGHT can carry one or many LUGGAGE items
Each LUGGAGE item is on one FLIGHT
- **BOOKING_DETAIL and PASSENGER**
One PASSENGER is included in one or many BOOKING_DETAIL(s)
Each BOOKING_DETAIL includes one PASSENGER

- **BOOKING and BOOKING_DETAIL**
One BOOKING can have one or many BOOKING_DETAIL(s)
Each BOOKING_DETAIL is specific to one BOOKING
- **FLIGHT and BOOKING_DETAIL**
One FLIGHT can have one or many BOOKING_DETAIL(s)
Each BOOKING_DETAIL is associated with one FLIGHT
- **BOOKING and BOOKING_PAYMENT**
One BOOKING must have one BOOKING_PAYMENT
Each BOOKING_PAYMENT belongs to one BOOKING
- **LUGGAGE and ADDITIONAL_CHARGE**
One LUGGAGE item can incur zero or many ADDITIONAL_CHARGE(s)
Each ADDITIONAL_CHARGE belongs to one LUGGAGE item
- **BOOKING_DETAIL and LUGGAGE**
One BOOKING_DETAIL has zero or many LUGGAGE items
Each LUGGAGE item belongs to one BOOKING_DETAIL

10. ER Diagram:



11. **Notes:**

Composite keys used in ERD

- BOOKING_DETAIL (Booking_ID, Passenger_ID, and Flight_ID together form the primary key)
- FLIGHT_CREW (Staff_ID and Flight_ID together form the primary key)
- FLIGHT_ASSIGNMENT (Staff_ID and Flight_ID together form the primary key)

Weak entities

- BOOKING_DETAIL
- FLIGHT_CREW
- FLIGHT_ASSIGNMENT

Bridge entities

- BOOKING_DETAIL
- FLIGHT_CREW
- FLIGHT_ASSIGNMENT

Strong entities

- BOOKING
- PASSENGER
- STAFF
- FLIGHT
- LUGGAGE
- ADDITIONAL_CHARGE
- AIRCRAFT
- ROUTE
- AIRPORT

Strong relationships

- STAFF and its sub-types (PILOT and FLIGHT_ATTENDANT)
- BOOKING and BOOKING_DETAIL
- FLIGHT and BOOKING_DETAIL

Weak relationships

- PASSENGER and BOOKING
- LUGGAGE and ADDITIONAL_CHARGE
- STAFF and FLIGHT_CREW
- FLIGHT and FLIGHT_CREW
- STAFF and FLIGHT_ASSIGNMENT
- FLIGHT and FLIGHT_ASSIGNMENT

Mandatory relationships

- BOOKING and BOOKING_DETAIL
- STAFF and its sub-types (PILOT and FLIGHT_ATTENDANT)

Optional relationships

- LUGGAGE and ADDITIONAL_CHARGE

Super entities

- STAFF

Sub-type entities

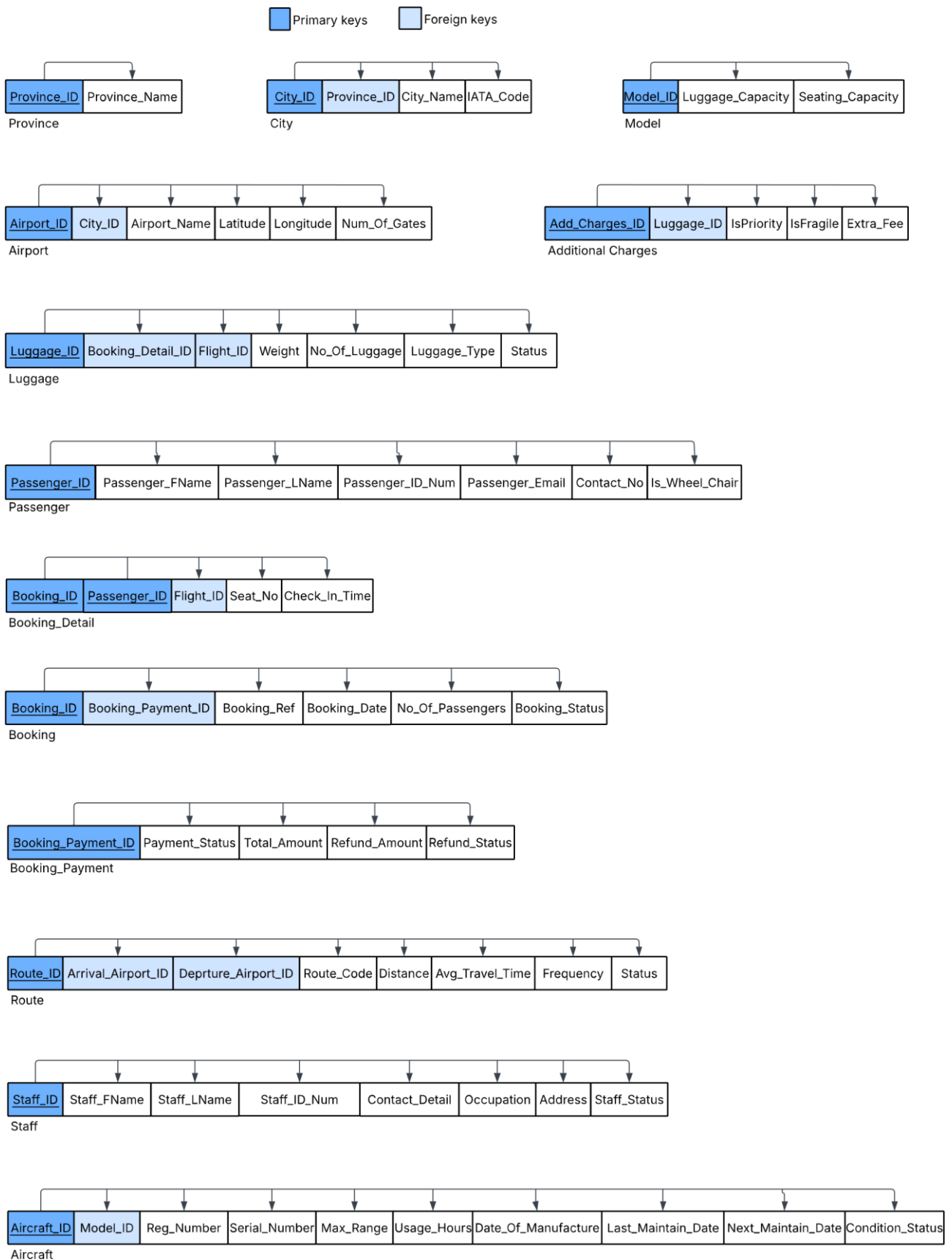
- PILOT
- FLIGHT_ATTENDANT

12. **Logical Design:**

- **PROVINCE**
(Province_ID(PK), Province_Name)
- **CITY**
(CITY_ID(PK), Province_ID(FK), City_Name, IATA_Code)
- **AIRPORT**
(Airport_ID(PK), CITY_ID(FK), Airport_Name, Latitude, Longitude, Number_Of_Gates)
- **MODEL**
(Model_ID(PK), Luggage_capacity, Seating_capacity)
- **AIRCRAFT**
(Aircraft_ID(PK), Model_ID(FK), Reg_number, Serial_number, Max_Range, Usage_Hours, Date_Of_Manufacture, Last_Maintain_Date, Next_Maintain_Date, Condition_Status)
- **ROUTE**
(Route_ID(PK), Arrival_Airport_ID(FK), Departure_Airport_ID(FK), Route_Code, Distance, Avg_Travel_Time, Frequency, Status)
- **FLIGHT**
(Flight_ID(PK), Aircraft_ID(FK), Route_ID(FK), Flight_Code, Departure_Date, Arrival_Date, Boarding_Time, Available_Seats, Flight_Status, Departure_Gate_No, Base_Price)
- **FLIGHT_ASSIGNMENT**
(Flight_ID(PK), Staff_ID(FK), Duty_start_time, Duty_end_time, Assignment_status)
- **FLIGHT_CREW**
(Staff_ID(PKFK), License_NO, Licence_Expiry_Date, Staff_Type)
- **STAFF**
(Staff_ID(PK), Staff_FName, Staff_LName, Staff_ID_Num, Contact_detail, Occupation, Address, Staff_Status)
- **FLIGHT_ATTENDENT**
(Staff_ID(PKFK), Service_Level, Years_of_Experience)

- **PILOT**
(Staff_ID(PKFK), Licence_Type, Rank, Flight_Hours)
- **ADDITIONAL_CHARGE**
(Add_Charges_ID, Luggage_ID(FK), isPriority, isFragile, Extra_Fee)
- **LUGGAGE**
(Luggage_ID(PK), Booking_Detail_ID(FK), Flight_ID(FK), Weight, No_Of_Luggage, Luggage_Type, Status)
- **PASSENGER**
(Passenger_ID(PK), Passenger_FName, Passenger_LName, Passenger_ID_Num, Passenger_email, Contact_no, isWheel_Chair)
- **BOOKING_DETAIL**
(Booking_ID(PKFK), Passenger_ID(PKFK), Flight_ID(FK), Seat_No, Check_In_Time)
- **BOOKING**
(Booking_ID(PK), Booking_Payment_ID(FK), Booking_Ref, Booking_Date, No_Of_Passengers, Booking_Status)
- **BOOKING_PAYMENT**
(Booking_Payment_ID(PK), Payment_Status, Total_Amount, Refund_Amount, Refund_Status)

13. Dependency diagrams:



<u>Staff_ID</u>	License_No	License_Expiry_Date	Staff_Type
-----------------	------------	---------------------	------------

Flight_Crew

<u>Flight_ID</u>	<u>Staff_ID</u>	Duty_Start_Time	Duty_End_Time	Assignment_Status
------------------	-----------------	-----------------	---------------	-------------------

Flight_Assignment

<u>Flight_ID</u>	<u>Aircraft_ID</u>	<u>Route_ID</u>	Flight_Code	Departure_Date	Arrival_Date	Boarding_Time	Available_Seats	Flight_Status	Departure_Gate_No	Base_Price
------------------	--------------------	-----------------	-------------	----------------	--------------	---------------	-----------------	---------------	-------------------	------------

Flight



CMPG311

PHASE 3:

PHYSICAL DESIGN

GROUP 18: 404 NOT FOUND

Delphy Stone (Group leader): 50509772

Mbali Mlambo: 50763458

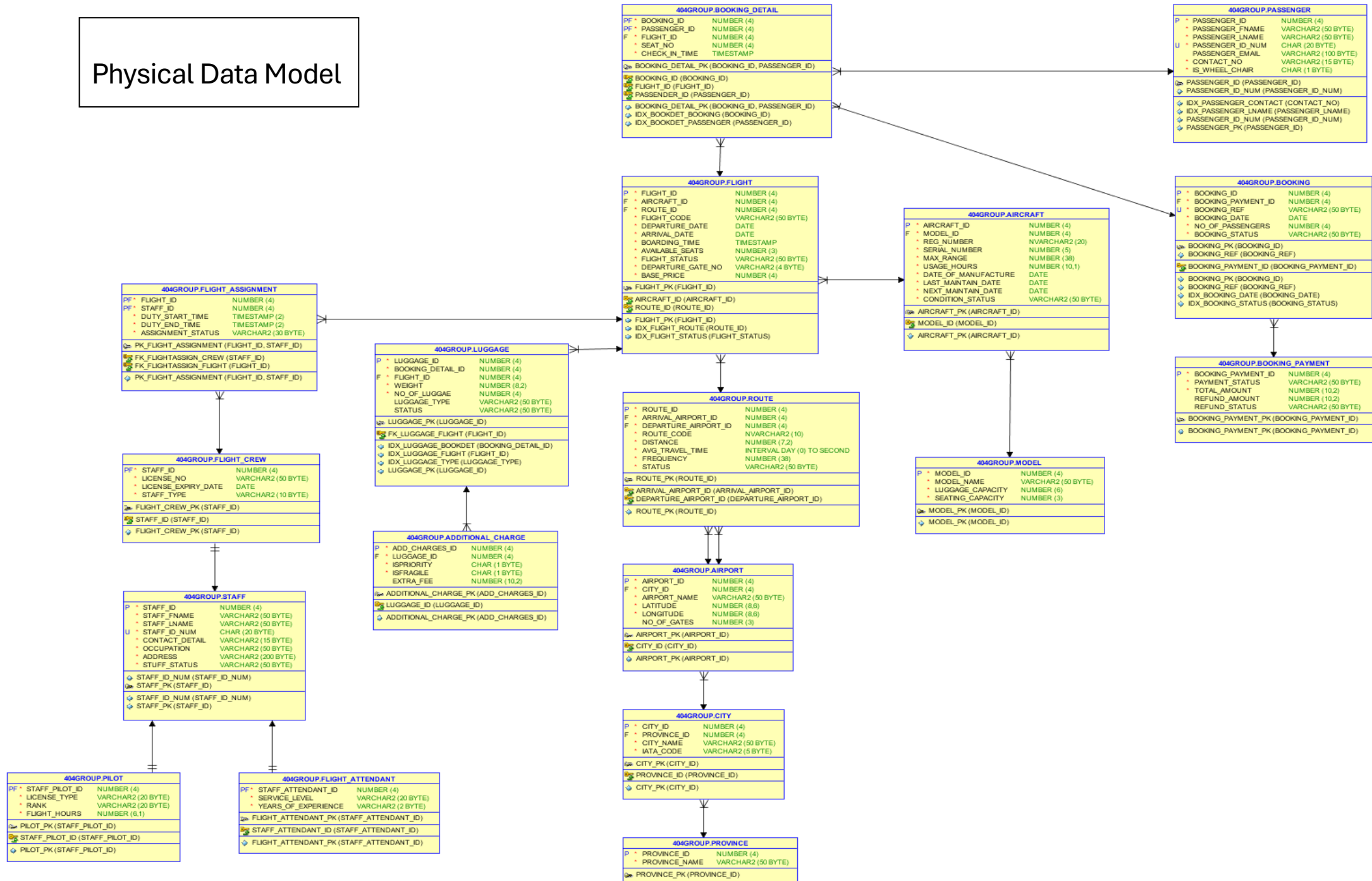
Tholoana Mothibi: 50371649

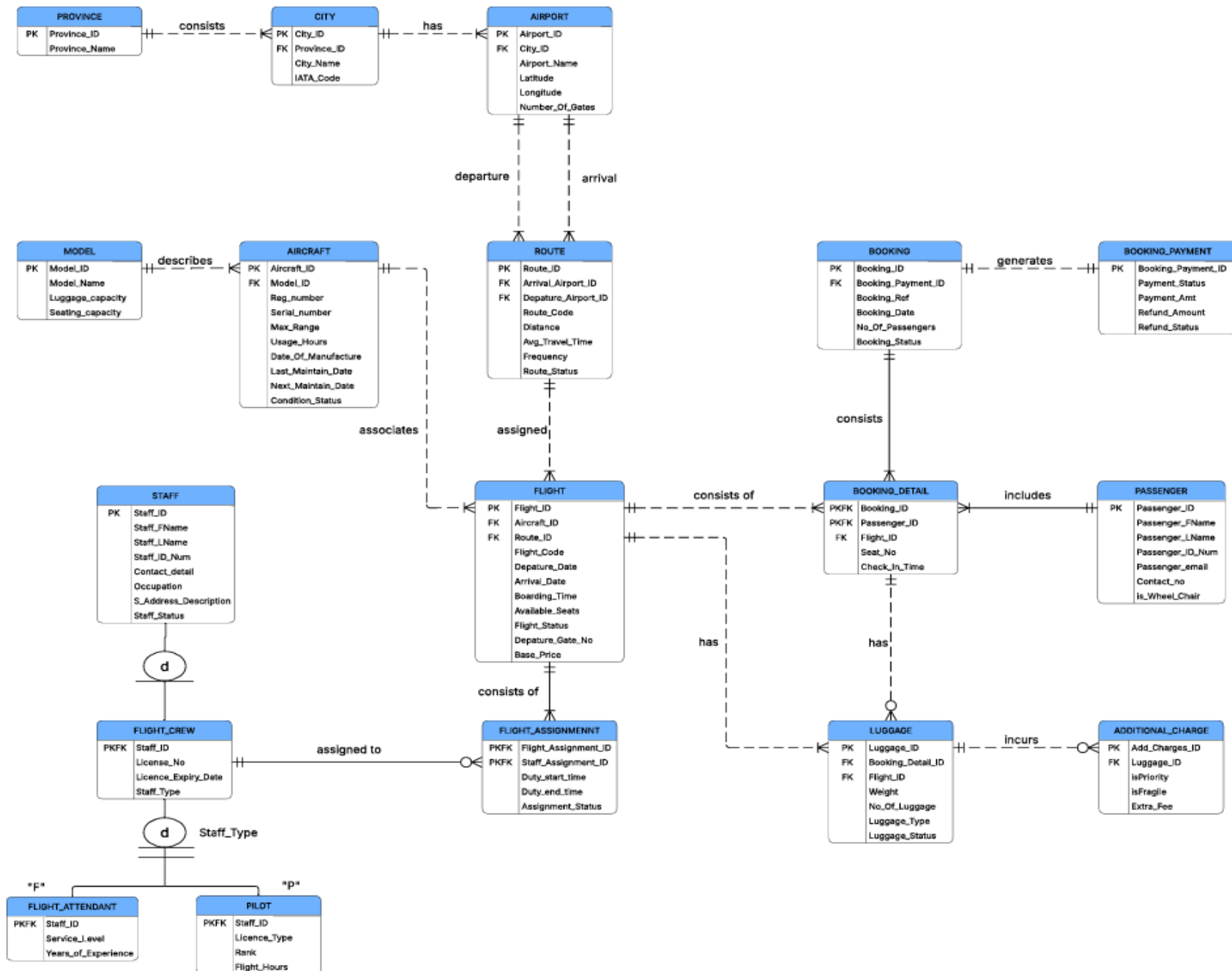
Itumeleng Maleka: 54122104

Khuthadzo Mudau: 51978156

Submission date: 2026/04/17

Physical Data Model





14. Database Objects

DROP TABLES

The DROP TABLE ... CASCADE CONSTRAINTS statement is used to permanently remove a table from the database together with all constraints that depend on it. The CASCADE CONSTRAINTS clause ensures that any foreign key constraints related to the table are also deleted automatically, preventing constraint errors during table deletion.

```
DROP TABLE ADDITIONAL_CHARGE CASCADE CONSTRAINTS;
DROP TABLE LUGGAGE CASCADE CONSTRAINTS;
DROP TABLE BOOKING_DETAIL CASCADE CONSTRAINTS;
DROP TABLE FLIGHT_ASSIGNMENT CASCADE CONSTRAINTS;
DROP TABLE PILOT CASCADE CONSTRAINTS;
DROP TABLE FLIGHT_ATTENDANT CASCADE CONSTRAINTS;
DROP TABLE FLIGHT_CREW CASCADE CONSTRAINTS;
DROP TABLE BOOKING CASCADE CONSTRAINTS;
DROP TABLE FLIGHT CASCADE CONSTRAINTS;
DROP TABLE STAFF CASCADE CONSTRAINTS;
DROP TABLE PASSENGER CASCADE CONSTRAINTS;
DROP TABLE ROUTE CASCADE CONSTRAINTS;
DROP TABLE AIRCRAFT CASCADE CONSTRAINTS;
DROP TABLE AIRPORT CASCADE CONSTRAINTS;
```

CREATE TABLES

Province table

PROVINCE table was designed as a relation where data on the various provinces in which the airports and cities are found would be stored. The Province_ID gives a unique identity to each province, while Province_Name holds the name of the province.

```
CREATE TABLE PROVINCE
(
    Province_ID    NUMBER(4) PRIMARY KEY,
    Province_Name  VARCHAR2(50) NOT NULL
);
```

City table

The table was designed to capture information related to cities, which are associated with particular provinces. The table features the city name and their airports'

IATA_Code. The foreign key relationship between CITY and PROVINCE was defined using Province_ID.

```
CREATE TABLE CITY
(
    City_ID          NUMBER(4) PRIMARY KEY,
    Province_ID      NUMBER(4) NOT NULL,
    City_Name        VARCHAR2(50) NOT NULL,
    IATA_Code        VARCHAR2(5) NOT NULL,
    FOREIGN KEY (Province_ID) REFERENCES PROVINCE(Province_ID)
);
```

Airport table

The table holds information related to airports based in various cities. It captures the airport name, geographical coordinates(Longitude and Latitude), and the number of gates in the airport. The foreign key relationship between the AIRPORT table CITY is defined using City_ID.

```
CREATE TABLE AIRPORT
(
    Airport_ID       NUMBER(4) PRIMARY KEY,
    City_ID          NUMBER(4) NOT NULL,
    Airport_Name      VARCHAR2(50) NOT NULL,
    Latitude          NUMBER(8,6) NOT NULL,
    Longitude         NUMBER(8,6) NOT NULL,
    Number_Of_Gates   NUMBER(1),
    FOREIGN KEY (City_ID) REFERENCES CITY(City_ID)
);
```

Route table

The ROUTE table was developed to capture flight routes details between airports. The table features arrival airport and departure airport, route code, travel distance, travel duration, flight frequency, and route status. Foreign keys were utilized to define relations between the AIRPORT table.


```

CREATE TABLE ROUTE
(
    Route_ID          NUMBER(4) PRIMARY KEY,
    Arrival_Airport_ID NUMBER(4) NOT NULL,
    Departure_Airport_ID NUMBER(4) NOT NULL,
    Route_Code        NVARCHAR2(10) NOT NULL,
    Distance           NUMBER(7,2) NOT NULL,
    Avg_Travel_Time    INTERVAL DAY(0) TO SECOND(0) NOT NULL,
    Frequency          NUMBER(50) NOT NULL,
    Status             VARCHAR2(50) NOT NULL,
    FOREIGN KEY (Arrival_Airport_ID) REFERENCES AIRPORT(Arrival_Airport_ID),
    FOREIGN KEY (Departure_Airport_ID) REFERENCES AIRPORT(Departure_Airport_ID)
);

```

Model table

The table contains information related to the various types of aircrafts that are used by SkyLink airline. Information related to the aircraft name, seat capacity, and luggage capacity can be found here.

```

CREATE TABLE MODEL
(
    Model_ID          NUMBER(4) PRIMARY KEY,
    Model_Name        VARCHAR2(50),
    Luggage_Capacity  NUMBER(6,2) NOT NULL,
    Seating_Capacity  NUMBER(3) NOT NULL
);

```

Aircraft table

The table was formed for the purpose of storing information about the aircrafts that belong to SkyLink airline. Some of the details are the registration number, serial number, range, hours of use, manufacturing date, and maintenance dates. The aircraft is associated with a particular aircraft through the foreign key Model_ID.

```

CREATE TABLE AIRCRAFT
(
    Aircraft_ID          NUMBER(4) PRIMARY KEY,
    Model_ID             NUMBER(4) NOT NULL,
    Reg_Number           NVARCHAR2(20) NOT NULL,
    Serial_Number        NUMBER(5) NOT NULL,
    Max_Range            NUMBER(40) NOT NULL,
    Usage_Hours          NUMBER(10,1) NOT NULL,
    Date_Of_Manufacture  DATE NOT NULL,
    Last_Maintain_Date   DATE NOT NULL,
    Next_Maintain_Date   DATE NOT NULL,
    Condition_Status     VARCHAR2(50) NOT NULL,
    FOREIGN KEY (Model_ID) REFERENCES MODEL(Model_ID)
);

```

Staff table

The table holds data on personal and professional information of all the employees in the airline. These attributes include the Staff_Name, Staff identification number, their contact information, occupation, address, and employment status

```

CREATE TABLE STAFF
(
    Staff_ID            NUMBER(4) PRIMARY KEY,
    Staff_FName         VARCHAR2(50) NOT NULL,
    Staff_LName         VARCHAR2(50) NOT NULL,
    Staff_ID_Num        CHAR(13) NOT NULL,
    Contact_Detail       VARCHAR2(15) NOT NULL,
    Occupation          VARCHAR2(50) NOT NULL,
    Address             VARCHAR2(200) NOT NULL,
    Staff_Status        VARCHAR2(50) NOT NULL
);

```

Flight crew table

The table captures the information pertaining to flight crew members who hold licenses. License details and expiry date along with the crew type information are captured. Each crew member is linked to the STAFF table through a foreign key relationship

```
CREATE TABLE FLIGHT_CREW
(
    Staff_ID          NUMBER(4) PRIMARY KEY,
    License_No        VARCHAR2(50) NOT NULL,
    License_Expiry_Date DATE NOT NULL,
    Staff_Type         VARCHAR2(10) NOT NULL,
    FOREIGN KEY (Staff_ID) REFERENCES STAFF(Staff_ID)
);
```

Pilot table

The table was formed as a sub-type of the STAFF table for recording pilot-specific data like license type, rank, and number of flight hours performed. The table uses the Staff_Pilot_ID to link itself with the STAFF table

```
CREATE TABLE PILOT
(
    Staff_Pilot_ID     NUMBER(4) PRIMARY KEY,
    License_Type        VARCHAR2(20) NOT NULL,
    Rank               VARCHAR2(20) NOT NULL,
    Flight_Hours        NUMBER(6,1) NOT NULL,
    FOREIGN KEY (Staff_Pilot_ID) REFERENCES STAFF(Staff_ID)
);
```

Flight attendant table

The table was formed to record flight attendant-specific information, such as service level and years of experience. The table relates to the STAFF table through the Staff_Flight_Attendant_ID.

```
CREATE TABLE FLIGHT_ATTENDANT
(
    Staff_Attendant_ID NUMBER(4) PRIMARY KEY,
    Service_Level        VARCHAR2(20) NOT NULL,
    Years_Of_Experience  NUMBER(2) NOT NULL,
    FOREIGN KEY (Staff_Attendant_ID) REFERENCES STAFF(Staff_ID)
);
```

Flight table

The table contains information pertaining to scheduled flights conducted by the airline. This table includes flight codes, departure and arrival dates and times, number of seats, status of the flight, departure gate numbers, and fare prices. The table is related to both the AIRCRAFT and ROUTE tables

```
CREATE TABLE FLIGHT
(
    Flight_ID          NUMBER(4) PRIMARY KEY,
    Aircraft_ID        NUMBER(4) NOT NULL,
    Route_ID           NUMBER(4) NOT NULL,
    Flight_Code         VARCHAR2(50) NOT NULL,
    Departure_Date      DATE NOT NULL,
    Arrival_Date        DATE NOT NULL,
    Boarding_Time       TIMESTAMP(2) NOT NULL,
    Available_Seats     NUMBER(3) NOT NULL,
    Flight_Status       VARCHAR2(50) NOT NULL,
    Departure_Gate_No   VARCHAR2(4) NOT NULL,
    Base_Price          NUMBER(4) NOT NULL,
    FOREIGN KEY (Aircraft_ID) REFERENCES AIRCRAFT(Aircraft_ID),
    FOREIGN KEY (Route_ID) REFERENCES ROUTE(Route_ID)
);
```

Flight assignment table

The table is created to keep track of the assignment of employees on the assigned flights. The table captures the starting and ending times of duties and their status.

```
CREATE TABLE FLIGHT_ASSIGNMENT
(
    Flight_Assignment_ID    NUMBER(4) NOT NULL,
    Staff_Assignment_ID     NUMBER(4) NOT NULL,
    Duty_Start_Time         TIMESTAMP(2) NOT NULL,
    Duty_End_Time           TIMESTAMP(2) NOT NULL,
    Assignment_Status       VARCHAR2(30) NOT NULL,

    PRIMARY KEY (Flight_Assignment_ID, Staff_ID),
    FOREIGN KEY (Flight_Assignment_ID) REFERENCES FLIGHT (Flight_ID),
    FOREIGN KEY (Staff_Assignment_ID) REFERENCES STAFF (Staff_ID)
);
```

Passenger table

The table is used to keep data such as passenger name, identity number, email address, phone number, address, and wheelchair information for the passenger. There was a

need to have a check constraint to have correct input on whether the passenger requires a wheelchair, Y for yes and N for no.

```
CREATE TABLE PASSENGER
(
    Passenger_ID          NUMBER(4) PRIMARY KEY,
    Passenger_FName       VARCHAR2(50) NOT NULL,
    Passenger_LName       VARCHAR2(50) NOT NULL,
    Passenger_ID_Num      CHAR(13) NOT NULL,
    Passenger_Email       VARCHAR2(100),
    Contact_No            VARCHAR2(15) NOT NULL,
    P_Adress_description  VARCHAR2(100),
    Is_Wheel_Chair        CHAR(1) CHECK (Is_Wheel_Chair IN ('Y','N'))
);
```

Booking payment table

The table is used to store all information about payments that have been made during bookings by passenger. Payment status, payment amount, refund amount and refund status can be found in this table

```
CREATE TABLE BOOKING_PAYMENT
(
    Booking_Payment_ID    NUMBER(4) PRIMARY KEY,
    Payment_Status        VARCHAR2(50) NOT NULL,
    Payment_amt           NUMBER(10,2) NOT NULL,
    Refund_Amount         NUMBER(10,2),
    Refund_Status         VARCHAR2(50)
);
```

Booking table

The table is used to store all the information regarding bookings made by the passenger, like booking reference, booking date, number of passengers, and booking status. The table references the BOOKING_PAYMENT table to connect bookings with payment records.

```

CREATE TABLE BOOKING
(
    Booking_ID          NUMBER(4) PRIMARY KEY,
    Booking_Payment_ID  NUMBER(4) NOT NULL,
    Booking_Ref         VARCHAR2(50) NOT NULL,
    Booking_Date        DATE NOT NULL,
    No_Of_Passengers    NUMBER(4) NOT NULL,
    Booking_Status      VARCHAR2(50) NOT NULL,
    FOREIGN KEY (Booking_Payment_ID) REFERENCES BOOKING_PAYMENT(Booking_Payment_ID)
);

```

Booking detail table

The table is used to create relationship between passengers, bookings and flight details. Details like seat allocation and passenger check in time are stored in this table and foreign keys have been used for relationships between booking, passenger and flight tables.

```

CREATE TABLE BOOKING_DETAIL
(
    Booking_ID          NUMBER(4) PRIMARY KEY,
    Passenger_ID        NUMBER(4) PRIMARY KEY,
    Flight_ID           NUMBER(4) NOT NULL,
    Seat_No             NUMBER(4) NOT NULL,
    Check_In_Time       TIMESTAMP(2),
    FOREIGN KEY (Booking_ID) REFERENCES BOOKING(Booking_ID),
    FOREIGN KEY (Passenger_ID) REFERENCES PASSENGER(Passenger_ID),
    FOREIGN KEY (Flight_ID) REFERENCES FLIGHT(Flight_ID)
);

```

Luggage table

The table is used to store details about the luggage carried by the passenger associated with the bookings and flights table. Details included in this table are luggage weight, luggage count, luggage type, and luggage status.

```

CREATE TABLE LUGGAGE
(
    Luggage_ID          NUMBER(4) PRIMARY KEY,
    Booking_Detail_ID    NUMBER(4) NOT NULL,
    Flight_ID           NUMBER(4) NOT NULL,
    Weight              NUMBER(8,2) NOT NULL,
    No_Of_Luggage        NUMBER(4) NOT NULL,
    Luggage_Type         VARCHAR2(50),
    Status              VARCHAR2(50),
    FOREIGN KEY (Booking_Detail_ID) REFERENCES BOOKING_DETAIL(Booking_Detail_ID),
    FOREIGN KEY (Flight_ID) REFERENCES FLIGHT(Flight_ID)
);

```

Additional charge table

The table has been created to store additional charges levied on passengers due to their luggage. The attributes isPriority and isFragile are validated using CHECK constraints to store either Y for yes or No for no values only

```
CREATE TABLE ADDITIONAL_CHARGE
(
    Add_Charge_ID    NUMBER(4) PRIMARY KEY,
    Luggage_ID       NUMBER(4) NOT NULL,
    isPriority        CHAR(1) CHECK (isPriority IN ('Y','N')),
    isFragile        CHAR(1) CHECK (isFragile IN ('Y','N')),
    Extra_Fee        NUMBER(10,2),
    FOREIGN KEY (Luggage_ID) REFERENCES LUGGAGE (Luggage_ID)
);
```

Populating tables (INSERT INTO SQL STATEMENTS)

INSERT INTO SQL statements are used to add new records into a table. These statements are essential for populating tables with data, supporting application functionality, and initializing database information when the tables are first created.

Table Names	INSERT SQL STATEMENTS
Province	<pre>INSERT INTO PROVINCE VALUES(3, 'Western Cape'); INSERT INTO PROVINCE VALUES(4, 'North West'); INSERT INTO PROVINCE VALUES(5, 'Free State'); INSERT INTO PROVINCE VALUES(6, 'KwaZulu-Natal'); INSERT INTO PROVINCE VALUES(7, 'Northern Cape'); INSERT INTO PROVINCE VALUES(8, 'Eastern Cape'); INSERT INTO PROVINCE VALUES(9, 'Mpumalanga');</pre>
City	<pre>INSERT INTO CITY VALUES(1,1, 'Johannesburg', 'JNB'); INSERT INTO CITY VALUES(2,1, 'Johannesburg', 'HLA'); INSERT INTO CITY VALUES(3,2, 'Polokwane', 'PTG'); INSERT INTO CITY VALUES(4,3, 'Cape Town', 'CPT'); INSERT INTO CITY VALUES(5,3, 'George', 'GRJ'); INSERT INTO CITY VALUES(6,5, 'Bloemfontein', 'BFN'); INSERT INTO CITY VALUES(7,6, 'Durban', 'DUR'); INSERT INTO CITY VALUES(8,7, 'Upington', 'UTN'); INSERT INTO CITY VALUES(9,7, 'Kimberley', 'KIM'); INSERT INTO CITY VALUES(10,8, 'Gqeberha', 'PLZ'); INSERT INTO CITY VALUES(11,8, 'Mthatha', 'UTT'); INSERT INTO CITY VALUES(12,8, 'East London', 'ELS'); INSERT INTO CITY VALUES(13,9, 'Mbombela', 'MQP');</pre>

Airport	<pre> INSERT INTO AIRPORT VALUES (1, 1, 'O.R. Tambo International', -26.1367, 28.2411, 65); INSERT INTO AIRPORT VALUES (2, 2, 'Lanseria International', -25.9385, 27.9261, 6); INSERT INTO AIRPORT VALUES (3, 3, 'Polokwane International', -23.8450, 29.4578, 4); INSERT INTO AIRPORT VALUES (4, 4, 'Cape Town International', -33.9715, 18.6021, 32); INSERT INTO AIRPORT VALUES (5, 5, 'George Airport', -34.0056, 22.3789, 8); INSERT INTO AIRPORT VALUES (6, 6, 'Bram Fischer International', -29.0927, 26.3024, 4); INSERT INTO AIRPORT VALUES (7, 7, 'King Shaka International', -29.6094, 31.1141, 16); INSERT INTO AIRPORT VALUES (8, 8, 'Upington International', -28.4014, 21.2706, 6); INSERT INTO AIRPORT VALUES (9, 9, 'Kimberley Airport', -28.8028, 24.7652, 6); INSERT INTO AIRPORT VALUES (10, 10, 'Chief Dawid Stuurman International', -33.9849, 25.6173, 6); INSERT INTO AIRPORT VALUES (11, 11, 'Mthatha Airport', -31.5476, 28.6738, 4); INSERT INTO AIRPORT VALUES (12, 12, 'King Phalo Airport', -33.0356, 27.8259, 4); INSERT INTO AIRPORT VALUES (13, 13, 'Kruger Mpumalanga International', -25.3849, 31.1105, 8); </pre>
Aircraft	<pre> -- INSERT INTO AIRCRAFT VALUES(1,1,'ZS-A04',1043,6300,12500, TO_DATE('2015-06-12','YYYY-MM-DD'),TO_DATE('2025-12-12','YYYY-MM-DD'), ADD_MONTHS(TO_DATE('2025-12-12','YYYY-MM-DD'),6),'Active'); -- INSERT INTO AIRCRAFT VALUES(2,2,'ZS-B01',35012,5765,10800, TO_DATE('2014-09-18','YYYY-MM-DD'),TO_DATE('2025-12-12','YYYY-MM-DD'), ADD_MONTHS(TO_DATE('2025-12-12','YYYY-MM-DD'),6),'Active'); -- INSERT INTO AIRCRAFT VALUES(3,3,'ZS-B09',65432,6570,8500, TO_DATE('2019-03-22','YYYY-MM-DD'),TO_DATE('2025-12-12','YYYY-MM-DD'), ADD_MONTHS(TO_DATE('2025-12-12','YYYY-MM-DD'),6),'Active'); -- INSERT INTO AIRCRAFT VALUES(4,1,'ZS-A06',1043,6300,12500, TO_DATE('2015-06-12','YYYY-MM-DD'),TO_DATE('2025-07-19','YYYY-MM-DD'), ADD_MONTHS(TO_DATE('2025-07-19','YYYY-MM-DD'),6),'Inactive'); -- INSERT INTO AIRCRAFT VALUES(5,1,'ZS-A03',1043,6300,12500, TO_DATE('2015-06-12','YYYY-MM-DD'),TO_DATE('2025-03-09','YYYY-MM-DD'), ADD_MONTHS(TO_DATE('2025-03-09','YYYY-MM-DD'),6),'Inactive'); -- INSERT INTO AIRCRAFT VALUES(6,2,'ZS-B08',35012,5765,10800, TO_DATE('2014-09-18','YYYY-MM-DD'),TO_DATE('2025-12-12','YYYY-MM-DD'), ADD_MONTHS(TO_DATE('2025-12-12','YYYY-MM-DD'),6),'Active'); -- INSERT INTO AIRCRAFT VALUES(7,2,'ZS-B03',35012,5765,10800, TO_DATE('2014-09-18','YYYY-MM-DD'),TO_DATE('2025-12-12','YYYY-MM-DD'), ADD_MONTHS(TO_DATE('2025-12-12','YYYY-MM-DD'),6),'Active'); -- INSERT INTO AIRCRAFT VALUES(8,2,'ZS-B05',35012,5765,10800, TO_DATE('2014-09-18','YYYY-MM-DD'),TO_DATE('2025-04-02','YYYY-MM-DD'), ADD_MONTHS(TO_DATE('2025-04-02','YYYY-MM-DD'),6),'Active'); </pre>

Route	
INSERT INTO ROUTE VALUES	(1, 1, 4, 'JNB-CPT', 1270, INTERVAL '0 2:15:00' DAY TO SECOND, 21, 'ACTIVE');
INSERT INTO ROUTE VALUES	(2, 1, 7, 'JNB-DUR', 555, INTERVAL '0 1:15:00' DAY TO SECOND, 18, 'ACTIVE');
INSERT INTO ROUTE VALUES	(3, 1, 6, 'JNB-BFN', 395, INTERVAL '0 1:05:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(4, 1, 10, 'JNB-PLZ', 740, INTERVAL '0 1:45:00' DAY TO SECOND, 10, 'ACTIVE');
INSERT INTO ROUTE VALUES	(5, 1, 8, 'JNB-UTN', 800, INTERVAL '0 1:35:00' DAY TO SECOND, 5, 'ACTIVE');
INSERT INTO ROUTE VALUES	(6, 1, 12, 'JNB-ELS', 920, INTERVAL '0 1:30:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(7, 1, 5, 'JNB-GRJ', 1145, INTERVAL '0 2:00:00' DAY TO SECOND, 6, 'ACTIVE');
INSERT INTO ROUTE VALUES	(8, 1, 9, 'JNB-KIM', 470, INTERVAL '0 1:05:00' DAY TO SECOND, 5, 'ACTIVE');
INSERT INTO ROUTE VALUES	(9, 1, 2, 'JNB-HLA', 39, INTERVAL '0 0:20:00' DAY TO SECOND, 14, 'ACTIVE');
INSERT INTO ROUTE VALUES	(10, 1, 13, 'JNB-MQP', 298, INTERVAL '0 0:55:00' DAY TO SECOND, 10, 'ACTIVE');
INSERT INTO ROUTE VALUES	(11, 1, 3, 'JNB-PTG', 283, INTERVAL '0 0:55:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(12, 1, 11, 'JNB-UTT', 710, INTERVAL '0 1:20:00' DAY TO SECOND, 5, 'ACTIVE');
INSERT INTO ROUTE VALUES	(13, 2, 1, 'CPT-JNB', 1270, INTERVAL '0 2:15:00' DAY TO SECOND, 21, 'ACTIVE');
INSERT INTO ROUTE VALUES	(14, 2, 7, 'CPT-DUR', 1272, INTERVAL '0 2:05:00' DAY TO SECOND, 14, 'ACTIVE');
INSERT INTO ROUTE VALUES	(15, 2, 6, 'CPT-BFN', 878, INTERVAL '0 1:35:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(16, 2, 10, 'CPT-PLZ', 740, INTERVAL '0 1:10:00' DAY TO SECOND, 10, 'ACTIVE');
INSERT INTO ROUTE VALUES	(17, 2, 8, 'CPT-UTN', 668, INTERVAL '0 1:15:00' DAY TO SECOND, 5, 'ACTIVE');
INSERT INTO ROUTE VALUES	(18, 2, 12, 'CPT-ELS', 1056, INTERVAL '0 1:35:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(19, 2, 5, 'CPT-GRJ', 395, INTERVAL '0 0:55:00' DAY TO SECOND, 9, 'ACTIVE');
INSERT INTO ROUTE VALUES	(20, 2, 9, 'CPT-KIM', 960, INTERVAL '0 1:35:00' DAY TO SECOND, 6, 'ACTIVE');
INSERT INTO ROUTE VALUES	(21, 2, 2, 'CPT-HLA', 1275, INTERVAL '0 2:05:00' DAY TO SECOND, 14, 'ACTIVE');
INSERT INTO ROUTE VALUES	(22, 2, 13, 'CPT-MQP', 1535, INTERVAL '0 2:20:00' DAY TO SECOND, 6, 'ACTIVE');
INSERT INTO ROUTE VALUES	(23, 2, 3, 'CPT-PTG', 1545, INTERVAL '0 2:25:00' DAY TO SECOND, 5, 'ACTIVE');
INSERT INTO ROUTE VALUES	(24, 2, 11, 'CPT-UTT', 1380, INTERVAL '0 2:10:00' DAY TO SECOND, 4, 'ACTIVE');
INSERT INTO ROUTE VALUES	(25, 3, 1, 'MQP-JNB', 298, INTERVAL '0 0:55:00' DAY TO SECOND, 21, 'ACTIVE');
INSERT INTO ROUTE VALUES	(26, 3, 4, 'MQP-CPT', 1535, INTERVAL '0 2:40:00' DAY TO SECOND, 6, 'ACTIVE');
INSERT INTO ROUTE VALUES	(27, 3, 7, 'MQP-DUR', 470, INTERVAL '0 1:05:00' DAY TO SECOND, 5, 'ACTIVE');
INSERT INTO ROUTE VALUES	(28, 3, 6, 'MQP-BFN', 629, INTERVAL '0 1:15:00' DAY TO SECOND, 3, 'ACTIVE');
INSERT INTO ROUTE VALUES	(29, 3, 10, 'MQP-PLZ', 910, INTERVAL '0 1:50:00' DAY TO SECOND, 3, 'ACTIVE');
INSERT INTO ROUTE VALUES	(30, 3, 8, 'MQP-UTN', 962, INTERVAL '0 1:55:00' DAY TO SECOND, 2, 'ACTIVE');
INSERT INTO ROUTE VALUES	(31, 3, 12, 'MQP-ELS', 840, INTERVAL '0 1:40:00' DAY TO SECOND, 2, 'ACTIVE');
INSERT INTO ROUTE VALUES	(32, 3, 5, 'MQP-GRJ', 1274, INTERVAL '0 2:30:00' DAY TO SECOND, 2, 'ACTIVE');
INSERT INTO ROUTE VALUES	(33, 3, 9, 'MQP-KIM', 720, INTERVAL '0 1:25:00' DAY TO SECOND, 2, 'ACTIVE');
INSERT INTO ROUTE VALUES	(34, 3, 2, 'MQP-HLA', 260, INTERVAL '0 0:50:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(35, 3, 3, 'MQP-PTG', 215, INTERVAL '0 0:45:00' DAY TO SECOND, 5, 'ACTIVE');
INSERT INTO ROUTE VALUES	(36, 3, 11, 'MQP-UTT', 723, INTERVAL '0 1:21:00' DAY TO SECOND, 2, 'ACTIVE');
INSERT INTO ROUTE VALUES	(37, 9, 1, 'KIM-JNB', 429, INTERVAL '0 01:25:00' DAY TO SECOND, 23, 'ACTIVE');
INSERT INTO ROUTE VALUES	(38, 9, 4, 'KIM-CPT', 818, INTERVAL '0 01:35:00' DAY TO SECOND, 18, 'ACTIVE');
INSERT INTO ROUTE VALUES	(39, 9, 7, 'KIM-DUR', 623.3, INTERVAL '0 02:10:00' DAY TO SECOND, 10, 'ACTIVE');
INSERT INTO ROUTE VALUES	(40, 9, 6, 'KIM-BFN', 143, INTERVAL '0 00:41:00' DAY TO SECOND, 5, 'ACTIVE');
INSERT INTO ROUTE VALUES	(41, 9, 10, 'KIM-PLZ', 586.46, INTERVAL '0 01:13:00' DAY TO SECOND, 8, 'ACTIVE');
INSERT INTO ROUTE VALUES	(42, 9, 8, 'KIM-UTN', 359, INTERVAL '0 01:04:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(78, 10, 3, 'PLZ-PTG', 1000, INTERVAL '0 02:15:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(79, 10, 2, 'PLZ-HLA', 900, INTERVAL '0 02:00:00' DAY TO SECOND, 11, 'ACTIVE');
INSERT INTO ROUTE VALUES	(80, 10, 11, 'PLZ-UTT', 300, INTERVAL '0 01:00:00' DAY TO SECOND, 8, 'ACTIVE');
INSERT INTO ROUTE VALUES	(81, 8, 11, 'UTN-UTT', 850, INTERVAL '0 01:50:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(82, 8, 10, 'UTN-PLZ', 950, INTERVAL '0 02:15:00' DAY TO SECOND, 8, 'ACTIVE');
INSERT INTO ROUTE VALUES	(83, 8, 4, 'UTN-CPT', 750, INTERVAL '0 01:45:00' DAY TO SECOND, 17, 'ACTIVE');
INSERT INTO ROUTE VALUES	(84, 8, 7, 'UTN-DUR', 900, INTERVAL '0 02:00:00' DAY TO SECOND, 10, 'ACTIVE');
INSERT INTO ROUTE VALUES	(85, 8, 1, 'UTN-JNB', 800, INTERVAL '0 01:50:00' DAY TO SECOND, 29, 'ACTIVE');
INSERT INTO ROUTE VALUES	(86, 8, 6, 'UTN-BFN', 480, INTERVAL '0 01:15:00' DAY TO SECOND, 28, 'ACTIVE');
INSERT INTO ROUTE VALUES	(87, 8, 5, 'UTN-GRJ', 700, INTERVAL '0 01:40:00' DAY TO SECOND, 24, 'ACTIVE');
INSERT INTO ROUTE VALUES	(88, 8, 12, 'UTN-ELS', 800, INTERVAL '0 01:50:00' DAY TO SECOND, 17, 'ACTIVE');
INSERT INTO ROUTE VALUES	(89, 8, 9, 'UTN-KIM', 400, INTERVAL '0 01:10:00' DAY TO SECOND, 7, 'ACTIVE');
INSERT INTO ROUTE VALUES	(90, 8, 13, 'UTN-MQP', 1300, INTERVAL '0 02:40:00' DAY TO SECOND, 27, 'ACTIVE');
INSERT INTO ROUTE VALUES	(91, 8, 3, 'UTN-PTG', 950, INTERVAL '0 02:10:00' DAY TO SECOND, 30, 'ACTIVE');
INSERT INTO ROUTE VALUES	(92, 8, 2, 'UTN-HLA', 820, INTERVAL '0 01:50:00' DAY TO SECOND, 27, 'ACTIVE');
INSERT INTO ROUTE VALUES	(93, 12, 1, 'ELS-JNB', 768, INTERVAL '0 1:20:00' DAY TO SECOND, 22, 'ACTIVE');
INSERT INTO ROUTE VALUES	(94, 12, 4, 'ELS-CPT', 861, INTERVAL '0 1:30:00' DAY TO SECOND, 18, 'ACTIVE');
INSERT INTO ROUTE VALUES	(95, 12, 7, 'ELS-DUR', 492, INTERVAL '0 1:00:00' DAY TO SECOND, 25, 'ACTIVE');
INSERT INTO ROUTE VALUES	(96, 12, 6, 'ELS-BFN', 462, INTERVAL '0 0:55:00' DAY TO SECOND, 12, 'ACTIVE');
INSERT INTO ROUTE VALUES	(97, 12, 10, 'ELS-PLZ', 230, INTERVAL '0 0:40:00' DAY TO SECOND, 30, 'ACTIVE');
INSERT INTO ROUTE VALUES	(98, 12, 8, 'ELS-UTN', 801, INTERVAL '0 1:25:00' DAY TO SECOND, 27, 'ACTIVE');
INSERT INTO ROUTE VALUES	(99, 12, 9, 'ELS-KIM', 554, INTERVAL '0 1:05:00' DAY TO SECOND, 14, 'ACTIVE');
INSERT INTO ROUTE VALUES	(100, 12, 2, 'ELS-HLA', 789, INTERVAL '0 1:25:00' DAY TO SECOND, 19, 'ACTIVE');
INSERT INTO ROUTE VALUES	(101, 12, 13, 'ELS-MQP', 908, INTERVAL '0 1:30:00' DAY TO SECOND, 33, 'ACTIVE');
INSERT INTO ROUTE VALUES	(102, 12, 3, 'ELS-PTG', 1034, INTERVAL '0 1:45:00' DAY TO SECOND, 28, 'ACTIVE');
INSERT INTO ROUTE VALUES	(103, 12, 11, 'ELS-UTT', 184, INTERVAL '0 0:35:00' DAY TO SECOND, 16, 'ACTIVE');
INSERT INTO ROUTE VALUES	(104, 12, 5, 'ELS-GRJ', 517, INTERVAL '0 1:10:00' DAY TO SECOND, 21, 'ACTIVE');

Flight	<pre> INSERT INTO FLIGHT VALUES (375, 3, 1, 'SA101', TO_DATE('2026-06-01', 'YYYY-MM-DD'), TO_DATE('2026-06-01', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-01 06:30:00', 'YYYY-MM-DD HH24:MI:SS'), 150, 'Scheduled', 'B8', 1500); INSERT INTO FLIGHT VALUES (88, 4, 4, 'SA202', TO_DATE('2026-06-02', 'YYYY-MM-DD'), TO_DATE('2026-06-02', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-02 08:30:00', 'YYYY-MM-DD HH24:MI:SS'), 180, 'Scheduled', 'A12', 1450); INSERT INTO FLIGHT VALUES (90, 5, 19, 'SA303', TO_DATE('2026-06-03', 'YYYY-MM-DD'), TO_DATE('2026-06-03', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-03 12:30:00', 'YYYY-MM-DD HH24:MI:SS'), 150, 'Scheduled', 'B67', 850); INSERT INTO FLIGHT VALUES (40, 2, 29, 'SA404', TO_DATE('2026-06-04', 'YYYY-MM-DD'), TO_DATE('2026-06-04', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-04 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), 200, 'Scheduled', 'B85', 1850); INSERT INTO FLIGHT VALUES (55, 1, 45, 'SA105', TO_DATE('2026-06-05', 'YYYY-MM-DD'), TO_DATE('2026-06-05', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-05 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), 180, 'Scheduled', 'A05', 1650); INSERT INTO FLIGHT VALUES (6, 5, 1, 'SA506', TO_DATE('2026-06-26', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-26-06 07:20:00', 'YYYY-MM-DD HH24:MI:SS'), TO_DATE('2026-06-26', 'YYYY-MM-DD'), 150, 'Scheduled', 'B02', 1200); </pre>
--------	--

	<pre> INSERT INTO FLIGHT VALUES (7, 5, 2, 'SA607', TO_DATE('2026-06-17', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-17 09:10:00', 'YYYY-MM-DD HH24:MI:SS'), TO_DATE('2026-06-17', 'YYYY-MM-DD'), 180, 'Scheduled', 'C01', 1400); INSERT INTO FLIGHT VALUES (8, 2, 3, 'SA208', TO_DATE('2026-06-28', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-28 11:45:00', 'YYYY-MM-DD HH24:MI:SS'), TO_DATE('2026-06-28', 'YYYY-MM-DD'), 200, 'Scheduled', 'A03', 1550); INSERT INTO FLIGHT VALUES (98, 4, 1, 'SA409', TO_DATE('2026-06-19', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-19 14:25:00', 'YYYY-MM-DD HH24:MI:SS'), TO_DATE('2026-06-19', 'YYYY-MM-DD'), 150, 'Scheduled', 'B04', 1300); INSERT INTO FLIGHT VALUES (101, 3, 2, 'SA310', TO_DATE('2026-06-10', 'YYYY-MM-DD'), TO_TIMESTAMP('2026-06-10 18:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_DATE('2026-06-10', 'YYYY-MM-DD'), 180, 'Scheduled', 'C03', 1100); </pre>
Flight Assignment	<pre> INSERT INTO FLIGHT_ASSIGNMENT VALUES (375, 1, TO_TIMESTAMP('2026-06-01 05:30:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2026-06-01 09:30:00', 'YYYY-MM-DD HH24:MI:SS'), 'Assigned'); INSERT INTO FLIGHT_ASSIGNMENT VALUES (88, 2, TO_TIMESTAMP('2026-06-01 05:30:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2026-06-01 09:30:00', 'YYYY-MM-DD HH24:MI:SS'), 'Assigned'); INSERT INTO FLIGHT_ASSIGNMENT VALUES (90, 4, TO_TIMESTAMP('2026-06-02 07:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2026-06-02 11:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Assigned'); INSERT INTO FLIGHT_ASSIGNMENT VALUES (40, 7, TO_TIMESTAMP('2026-06-02 07:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2026-06-02 11:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Assigned'); INSERT INTO FLIGHT_ASSIGNMENT VALUES (55, 5, TO_TIMESTAMP('2026-06-03 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2026-06-03 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Completed'); </pre>

Staff	<pre> INSERT INTO STAFF VALUES(1,'Itumeleng','Maleka','0606065422085','0764690382','Pilot','3570 Makhutso Drive Zone 3 Diepkloof','Inactive'); INSERT INTO STAFF VALUES(2,'Tholoana','Mothibi','0501085667184','0789231932','Flight attendant','12 Jacaranda St Pretoria','Active'); INSERT INTO STAFF VALUES(3,'Mhali','Mlambo','9804035567123','0782451008','Human resources','45 Main Rd Johannesburg','Active'); INSERT INTO STAFF VALUES(4,'Thabo','Nkosi','8501015800081','0824393798','Co-pilot','8 Oak Ave Bloemfontein','Active'); INSERT INTO STAFF VALUES(5,'Zanele','Dlamini','9203220045082','0738941240','Pilot','23 Beach Rd Durban','Inactive'); INSERT INTO STAFF VALUES(6,'Pieter','Van Der Merwe','7806154700083','0731234567','Flight Engineer','67 Protea St Soweto','Active'); INSERT INTO STAFF VALUES(7,'Nomsa','Zulu','9507180023084','0829876543','Senior flight attendant','3 Vineyard Rd Cape Town','Active'); INSERT INTO STAFF VALUES(8,'Sipho','Mhlangu','8812095600085','0748762134','Check-in agent','19 Hlubi St Pietermaritzburg','Active'); INSERT INTO STAFF VALUES(9,'Annika','Botha','9104225400086','0771561109','Gate agent','56 Gandhi Rd Lenasia','Active'); INSERT INTO STAFF VALUES(10,'Lungelo','Ntuli','8705016200087','0663331616','Aircraft maintenance engineer','34 Impala Rd Centurion','Inactive'); INSERT INTO STAFF VALUES(11,'Fatima','Patel','9309230180088','0761527001','Co-Pilot','78 Zulu St,Richards Bay','Active'); INSERT INTO STAFF VALUES(12,'Riaan','Joubert','8003154900089','0852689432','Pilot','88 Rooibos Ave George','Active'); INSERT INTO STAFF VALUES(13,'Luyanda','Mnatha','9775018287787','0663331616','Flight attendant','34 Impala Rd Centurion','Inactive'); INSERT INTO STAFF VALUES(14,'Thandeka','Mlambo','9309230181458','0761527001','Senior flight attendant','78 Zulu St,Richards Bay','Active'); INSERT INTO STAFF VALUES(15,'Sandile','Mda','8003157904529','0852689432','Flight attendant','88 Rooibos Ave George','Active'); </pre>
Flight crew	<pre> INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (1, 'ZA-PIL-001', TO_DATE('2027-06-15', 'YYYY-MM-DD'), 'P'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (2, 'ZA-ATT-002', TO_DATE('2026-03-20', 'YYYY-MM-DD'), 'F'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (4, 'ZA-PIL-004', TO_DATE('2028-01-10', 'YYYY-MM-DD'), 'P'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (5, 'ZA-PIL-005', TO_DATE('2026-09-30', 'YYYY-MM-DD'), 'P'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (6, 'ZA-ENG-006', TO_DATE('2027-11-25', 'YYYY-MM-DD'), 'P'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (7, 'ZA-ATT-007', TO_DATE('2026-07-14', 'YYYY-MM-DD'), 'F'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (11, 'ZA-PIL-011', TO_DATE('2029-02-28', 'YYYY-MM-DD'), 'P'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (12, 'ZA-PIL-012', TO_DATE('2027-08-19', 'YYYY-MM-DD'), 'P'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (13, 'ZA-ATT-013', TO_DATE('2027-03-15', 'YYYY-MM-DD'), 'F'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (14, 'ZA-ATT-014', TO_DATE('2026-11-20', 'YYYY-MM-DD'), 'F'); INSERT INTO FLIGHT_CREW (Staff_ID, License_No, License_Expiry_Date, Staff_Type) VALUES (15, 'ZA-ATT-015', TO_DATE('2028-05-10', 'YYYY-MM-DD'), 'F'); </pre>
Flight attendant	<pre> INSERT INTO FLIGHT_ATTENDANT (Staff_Attendant_ID, Service_Level, Years_of_Experience) VALUES (2, 'Economy', 3); INSERT INTO FLIGHT_ATTENDANT (Staff_Attendant_ID, Service_Level, Years_of_Experience) VALUES (7, 'Business', 8); INSERT INTO FLIGHT_ATTENDANT (Staff_Attendant_ID, Service_Level, Years_of_Experience) VALUES (13, 'Economy', 2); INSERT INTO FLIGHT_ATTENDANT (Staff_Attendant_ID, Service_Level, Years_of_Experience) VALUES (14, 'Business', 6); INSERT INTO FLIGHT_ATTENDANT (Staff_Attendant_ID, Service_Level, Years_of_Experience) VALUES (15, 'Economy', 1); </pre>
Pilot	<pre> INSERT INTO PILOT (Staff_Pilot_ID, License_Type, Rank, Flight_Hours) VALUES (1, 'ATPL', 'Captain', 5200); INSERT INTO PILOT (Staff_Pilot_ID, License_Type, Rank, Flight_Hours) VALUES (4, 'CPL', 'First Officer', 2800); INSERT INTO PILOT (Staff_Pilot_ID, License_Type, Rank, Flight_Hours) VALUES (5, 'ATPL', 'Captain', 6100); INSERT INTO PILOT (Staff_Pilot_ID, License_Type, Rank, Flight_Hours) VALUES (6, 'CPL', 'Flight Engineer', 3400); INSERT INTO PILOT (Staff_Pilot_ID, License_Type, Rank, Flight_Hours) VALUES (11, 'CPL', 'First Officer', 1900); INSERT INTO PILOT (Staff_Pilot_ID, License_Type, Rank, Flight_Hours) VALUES (12, 'ATPL', 'Captain', 7800); </pre>

Luggage	<pre> INSERT INTO LUGGAGE VALUES (1, 101, 1, 23.50, 2, 'Checked', 'Loaded-In'); INSERT INTO LUGGAGE VALUES (2, 102, 1, 7.00, 1, 'Carry-On', 'Checked-In'); INSERT INTO LUGGAGE VALUES (3, 103, 2, 30.75, 3, 'Oversized', 'Loaded-In'); INSERT INTO LUGGAGE VALUES (4, 104, 2, 8.00, 1, 'Carry-On', 'Checked-In'); INSERT INTO LUGGAGE VALUES (5, 105, 3, 20.00, 2, 'Checked', 'In Transit'); INSERT INTO LUGGAGE VALUES (6, 106, 4, 18.50, 2, 'Checked', 'Collected'); INSERT INTO LUGGAGE VALUES (7, 107, 142, 18.50, 2, 'Checked', 'Collected'); </pre>
Additional charges	<pre> INSERT INTO ADDITIONAL_CHARGE VALUES (1, 1, 'Y', 'N', 250.00); INSERT INTO ADDITIONAL_CHARGE VALUES (2, 2, 'N', 'N', 0.00); INSERT INTO ADDITIONAL_CHARGE VALUES (3, 3, 'Y', 'Y', 500.00); INSERT INTO ADDITIONAL_CHARGE VALUES (4, 4, 'N', 'Y', 150.00); INSERT INTO ADDITIONAL_CHARGE VALUES (5, 5, 'Y', 'N', 300.00); INSERT INTO ADDITIONAL_CHARGE VALUES (6, 6, 'N', 'N', 0.00); </pre>
Booking	<pre> INSERT INTO BOOKING VALUES (101,1,'B01409',TO_DATE('2026-05-01','YYYY-MM-DD'),1,'Confirmed'); INSERT INTO BOOKING VALUES (102,2,'B03267',TO_DATE('2026-05-02','YYYY-MM-DD'),2,'Confirmed'); INSERT INTO BOOKING VALUES (103,3,'B09873',TO_DATE('2026-05-03','YYYY-MM-DD'),1,'Pending'); INSERT INTO BOOKING VALUES (104,4,'B08342',TO_DATE('2026-05-04','YYYY-MM-DD'),1,'Confirmed'); INSERT INTO BOOKING VALUES (105,5,'B07511',TO_DATE('2026-05-05','YYYY-MM-DD'),1,'Confirmed'); INSERT INTO BOOKING VALUES (106,6,'B07512',TO_DATE('2026-05-05','YYYY-MM-DD'),1,'Confirmed'); INSERT INTO BOOKING VALUES (107,6,'B07515',TO_DATE('2026-12-05','YYYY-MM-DD'),1,'Confirmed'); </pre>
Booking payment	<pre> INSERT INTO BOOKING_PAYMENT VALUES (1,'Paid',2500.00,0,'None'); INSERT INTO BOOKING_PAYMENT VALUES (2,'Paid',3800.00,0,'None'); INSERT INTO BOOKING_PAYMENT VALUES (3,'Pending',2200.00,0,'None'); INSERT INTO BOOKING_PAYMENT VALUES (4,'Paid',4100.00,0,'None'); INSERT INTO BOOKING_PAYMENT VALUES (5,'Paid',2500.00,0,'None'); INSERT INTO BOOKING_PAYMENT VALUES (6,'Paid',2500.00,0,'None'); INSERT INTO BOOKING_PAYMENT VALUES (7,'Paid',4100.00,0,'None'); </pre>
Passenger	<pre> INSERT INTO PASSENGER VALUES (1,'Sipho','Dlamini','9001015009087','sipho@email.com','0821234567','N'); INSERT INTO PASSENGER VALUES (2,'Nomsa','Khumalo','9205200234081','Nomsa@email.com','0731234567','N'); INSERT INTO PASSENGER VALUES (3,'James','van der Berg','8811305001082','James@email.com','0611234567','Y'); INSERT INTO PASSENGER VALUES (4,'Aisha','Patel','9507152345089','Aisha@email.com','0841234567','N'); INSERT INTO PASSENGER VALUES (5,'Thabo','Mokoena','8904205678086','Thabo@email.com','0791234567','Y'); INSERT INTO PASSENGER VALUES (6,'Thabo','Bester','8988205678086','Thabo@email.com','0791234567','Y'); </pre>
Booking detail	<pre> INSERT INTO BOOKING_DETAIL (Booking_ID, Passenger_ID, Flight_ID, Seat_No, Check_In_Time) VALUES (101, 1, 375, 12, TO_TIMESTAMP('2026-06-01 05:00:00', 'YYYY-MM-DD HH24:MI:SS')); INSERT INTO BOOKING_DETAIL (Booking_ID, Passenger_ID, Flight_ID, Seat_No, Check_In_Time) VALUES (102, 2, 88, 14, TO_TIMESTAMP('2026-06-01 05:15:00', 'YYYY-MM-DD HH24:MI:SS')); INSERT INTO BOOKING_DETAIL (Booking_ID, Passenger_ID, Flight_ID, Seat_No, Check_In_Time) VALUES (103, 3, 90, 22, TO_TIMESTAMP('2026-06-02 06:30:00', 'YYYY-MM-DD HH24:MI:SS')); INSERT INTO BOOKING_DETAIL (Booking_ID, Passenger_ID, Flight_ID, Seat_No, Check_In_Time) VALUES (104, 4, 40, 35, TO_TIMESTAMP('2026-06-02 06:45:00', 'YYYY-MM-DD HH24:MI:SS')); INSERT INTO BOOKING_DETAIL (Booking_ID, Passenger_ID, Flight_ID, Seat_No, Check_In_Time) VALUES (105, 5, 55, 8, TO_TIMESTAMP('2026-06-03 07:00:00', 'YYYY-MM-DD HH24:MI:SS')); INSERT INTO BOOKING_DETAIL (Booking_ID, Passenger_ID, Flight_ID, Seat_No, Check_In_Time) VALUES (106, 6, 55, 9, TO_TIMESTAMP('2026-06-03 07:00:00', 'YYYY-MM-DD HH24:MI:SS')); INSERT INTO BOOKING_DETAIL (Booking_ID, Passenger_ID, Flight_ID, Seat_No, Check_In_Time) VALUES (107, 6, 142, 15, TO_TIMESTAMP('2026-12-31 08:00:00', 'YYYY-MM-DD HH24:MI:SS')); </pre>

UPDATE TABLE

This query updates Flight_ID = 375 and 145 statuses from scheduled to cancelled.

```
-- Update Flight 375
UPDATE FLIGHT
SET Flight_status = 'Cancelled'
WHERE Flight_ID = 375;

-- Update Flight 145
UPDATE FLIGHT
SET Flight_status = 'Cancelled'
WHERE Flight_ID = 145;
```

CREATE VIEWS

CREATE VIEW statements are used to create virtual tables based on the result of a SQL query. Views help simplify complex queries, improve data organization, and provide easier access to frequently used information without storing duplicate data in the database.

Flight detail view:

This view combines staff, flight crew, and flight attendant information into a single unified record for each flight attendant. It joins the STAFF, FLIGHT_CREW, and FLIGHT_ATTENDANT tables to retrieve each attendant's personal details such as name, contact information, and address alongside their professional details including license number, license expiry date, service level, and years of experience. This view serves as a convenient reference for managing and reviewing complete flight attendant profiles in one place.

CREATE OR REPLACE VIEW FLIGHTATTENDANTDETAILVIEW AS

SELECT

s.STAFF_ID,
s.STAFF_FNAME,
s.STAFF_LNAME,
s.CONTACT_DETAIL,
s.ADDRESS,
s.STUFF_STATUS,
fc.LICENSE_NO,
fc.LICENSE_EXPIRY_DATE,
fa.SERVICE_LEVEL,
fa.YEARS_OF_EXPERIENCE

FROM STAFF s


```
JOIN FLIGHT_CREW fc ON s.STAFF_ID = fc.STAFF_ID
JOIN FLIGHT_ATTENDANT fa ON fc.STAFF_ID = fa.STAFF_ATTENDANT_ID;
```

Booking detail view:

This view combines pilot and assignment information into a single unified record for each pilot. It joins the STAFF, FLIGHT_CREW, PILOT, and FLIGHT_ASSIGNMENT tables to retrieve each pilot's personal details such as name, alongside their professional details including rank and total flight hours, as well as their assignment information covering flight assignment ID, duty start time, and duty end time. This view provides a convenient overview of pilot schedules and qualifications for flight assignment management and planning purposes.

CREATE OR REPLACE VIEW BOOKINGDETAILVIEW AS

SELECT

```
  b.BOOKING_ID,
  b.BOOKING_REF,
  b.BOOKING_DATE,
  b.NO_OF_PASSENGERS,
  b.BOOKING_STATUS,
  bd.SEAT_NO,
  bd.CHECK_IN_TIME,
  p.PASSENGER_ID,
  p.PASSENGER_FNAME,
  p.PASSENGER_LNAME,
  p.CONTACT_NO,
  f.FLIGHT_ID,
  f.FLIGHT_CODE,
  f.DEPARTURE_DATE,
  f.ARRIVAL_DATE,
  f.DEPARTURE_GATE_NO,
  f.BASE_PRICE
```

FROM BOOKING b

JOIN BOOKING_DETAIL bd ON b.BOOKING_ID = bd.BOOKING_ID

JOIN PASSENGER p ON bd.PASSENGER_ID = p.PASSENGER_ID

JOIN FLIGHT f ON bd.FLIGHT_ID = f.FLIGHT_ID;

Pilot assignment view:

This view combines pilot and assignment information into a single unified record for each pilot. It joins the STAFF, FLIGHT_CREW, PILOT, and FLIGHT_ASSIGNMENT tables to retrieve each pilot's personal details such as name, alongside their professional details including rank and total flight hours, as well as their assignment information covering flight assignment ID, duty start time, and duty end time. This view provides a convenient overview of pilot schedules and qualifications for flight assignment management and planning purposes.

CREATE OR REPLACE VIEW PILOTASSIGNMENTVIEW AS

SELECT

s.STAFF_FNAME,
s.STAFF_LNAME,
p.RANK,
p.FLIGHT_HOURS,
fa.FLIGHT_ASSIGNMENT_ID,
fa.DUTY_START_TIME,
fa.DUTY_END_TIME

FROM STAFF s

JOIN FLIGHT_CREW fc ON s.STAFF_ID = fc.STAFF_ID

JOIN PILOT p ON fc.STAFF_ID = p.STAFF_PILOT_ID

JOIN FLIGHT_ASSIGNMENT fa ON p.STAFF_PILOT_ID = fa.STAFF_ASSIGNMENT_ID;

Flight detail view:

This view joins the FLIGHT, AIRCRAFT, ROUTE, and AIRPORT tables to produce a complete overview of each flight. It combines flight scheduling details, aircraft information, and route data including departure and arrival airport names, distance, and average travel time into a single convenient reference for flight operations and reporting.

CREATE OR REPLACE VIEW FLIGHTDETAILVIEW AS

SELECT

f.FLIGHT_ID,
f.FLIGHT_CODE,
f.DEPARTURE_DATE,
f.ARRIVAL_DATE,
f.DEPARTURE_GATE_NO,
f.BASE_PRICE,
f.AVAILABLE_SEATS,
a.AIRCRAFT_ID,
a.REG_NUMBER,
a.CONDITION_STATUS,

```
r.ROUTE_CODE,  
r.DISTANCE,  
r.AVG_TRAVEL_TIME,  
dep.AIRPORT_NAME AS DEPARTURE_AIRPORT,  
arr.AIRPORT_NAME AS ARRIVAL_AIRPORT  
FROM FLIGHT f  
JOIN AIRCRAFT a ON f.AIRCRAFT_ID = a.AIRCRAFT_ID  
JOIN ROUTE r ON f.ROUTE_ID = r.ROUTE_ID  
JOIN AIRPORT dep ON r.DEPARTURE_AIRPORT_ID = dep.AIRPORT_ID  
JOIN AIRPORT arr ON r.ARRIVAL_AIRPORT_ID = arr.AIRPORT_ID;
```

CREATE INDEX

Index creation

Indexes were created on frequently searched columns within the database to improve query performance and speed up data retrieval operations. These indexes help the database locate records more efficiently when executing searches, filtering data, or performing joins between tables.

Passenger Indexes

The idx_passenger_lname index was created on the PASSENGER_LNAME attribute in the PASSENGER table. This index improves the speed of searching and retrieving passenger records using surnames. It is useful when staff need to quickly locate passengers by their last names during bookings, check-ins, or report generation.

The idx_passenger_idnum index was created on the PASSANGER_ID_NUM attribute in the PASSENGER table. This index helps speed up searches using passenger identification numbers. It allows the system to quickly identify and retrieve passenger information for verification and booking purposes.

The idx_passenger_contact index was created on the CONTACT_NO attribute in the PASSENGER table. This index improves the performance of searches involving passenger contact numbers. It is useful when staff need to contact passengers or retrieve customer information efficiently.

```
/*PASSENGER*/  
CREATE INDEX idx_passenger_lname  
ON PASSENGER(PASSENGER_LNAME);  
  
CREATE INDEX idx_passenger_idnum  
ON PASSENGER(PASSANGER_ID_NUM);  
  
CREATE INDEX idx_passenger_contact  
ON PASSENGER(CONTACT_NO);
```

Flight indexes

The idx_flight_deptime index was created on the DEPARTURE_DATE attribute in the FLIGHT table. This index improves the speed of retrieving flights scheduled for a particular departure date. It is useful for flight scheduling, booking searches, and operational planning.

The idx_flight_status index was created on the FLIGHT_STATUS attribute in the FLIGHT table. This index helps speed up the filtering of flights according to their status, such as

delayed, cancelled, or active flights. It assists airport staff in monitoring and managing flight operations more efficiently.

The idx_flight_route index was created on the ROUTE_ID attribute in the FLIGHT table. This index improves searches for flights operating on specific routes. It is useful for route management, destination searches, and generating flight reports.

```
/*FLIGHT*/  
CREATE INDEX idx_flight_depdate  
ON FLIGHT(DEPARTURE_DATE);  
  
CREATE INDEX idx_flight_status  
ON FLIGHT(FLIGHT_STATUS);  
  
CREATE INDEX idx_flight_route  
ON FLIGHT(ROUTE_ID);
```

Booking indexes

The idx_booking_ref index was created on the BOOKING_REF attribute in the BOOKING table. This index speeds up searches using booking reference numbers. It allows staff to quickly retrieve booking records for customer support and booking confirmation purposes.

The idx_booking_status index was created on the BOOKING_STATUS attribute in the BOOKING table. This index improves the performance of filtering bookings based on their status, such as confirmed, pending, or cancelled bookings. It helps with booking management and reporting.

The idx_booking_date index was created on the BOOKING_DATE attribute in the BOOKING table. This index improves searches involving booking dates and helps generate reports for bookings made within specific periods.

```
/* BOOKING*/  
CREATE INDEX idx_booking_ref  
ON BOOKING(BOOKING_REF);  
  
CREATE INDEX idx_booking_status  
ON BOOKING(BOOKING_STATUS);  
  
CREATE INDEX idx_booking_date  
ON BOOKING(BOOKING_DATE);
```

Booking_detail indexes

The idx_bookdet_booking index was created on the BOOKING_ID attribute in the BOOKING_DETAIL table. This index improves the speed of retrieving booking detail records associated with a specific booking. It is useful when joining booking information with passenger and flight data.

The idx_bookdet_passenger index was created on the PASSENGER_ID attribute in the BOOKING_DETAIL table. This index speeds up searches for booking details linked to a passenger. It helps retrieve all bookings associated with a specific passenger efficiently.

The idx_bookdet_flight index was created on the FLIGHT_ID attribute in the BOOKING table. This index improves retrieval of bookings associated with a particular flight. It is useful for generating passenger manifests and flight-related reports.

```
/*BOOKING_DETAIL*/  
CREATE INDEX idx_bookdet_booking  
ON BOOKING_DETAIL(BOOKING_ID);  
  
CREATE INDEX idx_bookdet_passenger  
ON BOOKING_DETAIL(PASSENGER_ID);  
  
CREATE INDEX idx_bookdet_flight  
ON BOOKING(FLIGHT_ID);
```

Luggage indexes

The idx_luggage_bookdet index was created on the BOOKING_DETAIL_ID attribute in the LUGGAGE table. This index improves the speed of retrieving luggage records connected to booking details. It helps manage passenger baggage information efficiently.

The idx_luggage_flight index was created on the FLIGHT_ID attribute in the LUGGAGE table. This index speeds up searches for luggage linked to specific flights. It is useful for baggage tracking and flight baggage management.

The idx_luggage_type index was created on the LUGGAGE_TYPE attribute in the LUGGAGE table. This index improves filtering and searching luggage records according to luggage type. It helps separate and manage different categories of baggage more efficiently.

```
/* LUGGAGE*/  
CREATE INDEX idx_luggage_bookdet  
ON LUGGAGE(BOOKING_DETAIL_ID);  
  
CREATE INDEX idx_luggage_flight  
ON LUGGAGE(FLIGHT_ID);  
  
CREATE INDEX idx_luggage_type  
ON LUGGAGE(LUGGAGE_TYPE);  
  
SELECT index_name, table_name, status  
FROM user_indexes  
WHERE index_name LIKE 'IDX%';
```

QUERIES

SQL statements and reports

Summary of usage

Components	Statements
Limitation of rows and columns	Statement 15
Sorting	Statement 1,2, 3, 4, 5, 6, 15
LIKE, AND and OR	Statement 1,2
Variables and character functions	Statement 3, 15, 17
Round or trunc	Statement 10, 11
Date functions	Statement 15, 17
Aggregate functions	Statement 2,3, 4, 5, 8, 9, 10, 12, 13, 14, 17
Group by and having	Statement 3, 4, 5, 8, 9, 10,17
Joins	Statement 1, 2, 3, 4, 5, 7, 9
Sub-queries	Statement 2, 12, 13, 14
Extra functionality	Triggers – Statement 16 Cases – Statement 17

Statement 1

This query displays all cancelled flights on Johannesburg (JNB) routes that still have more than 100 available seats, retrieved by joining the FLIGHT and ROUTE tables.

```

/*View flights on the JNB route that were cancelled*/
SELECT *
FROM FLIGHT f
JOIN ROUTE r
  ON f.ROUTE_ID = r.ROUTE_ID
WHERE ROUTE_CODE LIKE 'JNB%'
AND FLIGHT_STATUS = 'Cancelled'
AND AVAILABLE_SEATS > 100;

```

FLIGHT_ID	AIRCRAFT_ID	ROUTE_ID	FLIGHT_CODE	DEPARTURE	ARRIVAL_D	BOARDING_TIME	AVAILABLE_SEATS	FLIGHT_STATUS
145	2	1	SA786	29/MAR/27	29/MAR/27	29/MAR/27 18:10:00.000000000	150	Cancelled
375	3	1	SA101	01/JUN/26	01/JUN/26	01/JUN/26 06:30:00.000000000	150	Cancelled

DEPA	BASE_PRICE	ROUTE_ID	ARRIVAL_AIRPORT_ID	DEPARTURE_AIRPORT_ID	ROUTE_CODE	DISTANCE	AVG_TRAVEL_TIME	FREQUENCY	STATUS
A1	1440	1	1	4	JNB-CPT	1270	+00 02:15:00.000000	26	ACTIVE
B8	1500	1	1	4	JNB-CPT	1270	+00 02:15:00.000000	26	ACTIVE

Statement 2

This query makes use of a view that displays the most popular routes for each aircraft, together with the number of flights and passengers flown on each route, as well as the airports of departure and arrival. The AIRCRAFT, MODEL, ROUTE, FLIGHT, BOOKING_DETAIL and AIRPORT databases are joined to get data, which is then sorted by aircraft and usage frequency.

```

/*REPORT: MOST USED ROUTE PER AIRCRAFT*/
CREATE OR REPLACE VIEW V_AIRCRAFT_ROUTE_USAGE AS
SELECT
    A.AIRCRAFT_ID,
    M.MODEL_ID,
    R.ROUTE_ID,
    R.ROUTE_CODE,
    (SELECT COUNT(F2.FLIGHT_ID)
     FROM FLIGHT F2
     WHERE F2.AIRCRAFT_ID = A.AIRCRAFT_ID
     AND F2.ROUTE_ID = R.ROUTE_ID) AS TIMES_FLOWN,
    (SELECT COUNT(BD.PASSENGER_ID)
     FROM BOOKING_DETAIL BD
     JOIN FLIGHT F3 ON BD.FLIGHT_ID = F3.FLIGHT_ID
     WHERE F3.AIRCRAFT_ID = A.AIRCRAFT_ID
     AND F3.ROUTE_ID = R.ROUTE_ID) AS TOTAL_PASSENGERS,
    (SELECT AP.AIRPORT_NAME
     FROM AIRPORT AP
     WHERE AP.AIRPORT_ID = R.DEPARTURE_AIRPORT_ID) AS DEPARTURE_AIRPORT,
    (SELECT AP.AIRPORT_NAME
     FROM AIRPORT AP
     WHERE AP.AIRPORT_ID = R.ARRIVAL_AIRPORT_ID) AS ARRIVAL_AIRPORT
FROM AIRCRAFT A
JOIN MODEL M ON A.MODEL_ID = M.MODEL_ID
JOIN ROUTE R ON R.ROUTE_ID IN (
    SELECT DISTINCT F5.ROUTE_ID
    FROM FLIGHT F5
    WHERE F5.AIRCRAFT_ID = A.AIRCRAFT_ID
)
ORDER BY A.AIRCRAFT_ID, TIMES_FLOWN DESC;

```

AIRCRAFT_ID	MODEL_ID	ROUTE_ID	ROUTE_CODE	TIMES_FLOWN	TOTAL_PASSENGERS	DEPARTURE_AIRPORT	ARRIVAL_AIRPORT
1	1	30 MQP-UTN		1	0	Upington International	Polokwane International
1	1	25 MQP-JNB		1	0	O.R. Tambo International	Polokwane International
1	1	40 KIM-BFN		1	0	Bram Fischer International	Kimberley Airport
1	1	35 MQP-PTG		1	0	Polokwane International	Polokwane International
1	1	20 CPT-KIM		1	0	Kimberley Airport	Lanseria International
1	1	45 KIM-HLA		1	2	Lanseria International	Kimberley Airport
2	2	1 JNB-CPT		2	0	Cape Town International	O.R. Tambo International
2	2	26 MQP-CPT		1	0	Cape Town International	Polokwane International
2	2	31 MQP-ELS		1	0	King Phalo Airport	Polokwane International
2	2	41 KIM-PLZ		1	0	Chief Dawid Stuurman International	Kimberley Airport
2	2	36 MQP-UTT		1	0	Mthatha Airport	Polokwane International
2	2	29 MQP-PLZ		1	1	Chief Dawid Stuurman International	Polokwane International
2	2	21 CPT-HLA		1	0	Lanseria International	Lanseria International
3	3	27 MQP-DUR		1	0	King Shaka International	Polokwane International
3	3	22 CPT-MQP		1	0	Kruger Mpumalanga International	Lanseria International
3	3	1 JNB-CPT		1	1	Cape Town International	O.R. Tambo International
3	3	42 KIM-UTN		1	0	Upington International	Kimberley Airport
3	3	37 KIM-JNB		1	0	O.R. Tambo International	Kimberley Airport
3	3	32 MQP-GRJ		1	0	George Airport	Polokwane International
4	1	30 MQP-UTN		1	0	Bram Fischer International	Polokwane International

Statement 3

This query creates a view called VW_POPULAR_DESTINATIONS to generate a report showing the most popular flight destinations based on the number of bookings made. It uses the COUNT() aggregate function to calculate the total number of bookings for each destination airport. Multiple tables that are joined are BOOKING, BOOKING_DETAIL, FLIGHT, ROUTE and AIRPORT are joined together to retrieve the required information. A rank function was used to rank destinations from most booked to least booked according to the total bookings.

```

/*most popular destination report*/
CREATE VIEW vw_POPULAR_destinations AS
SELECT INITCAP(a.AIRPORT_NAME) AS DESTINATION,
       COUNT(b.BOOKING_ID) AS TOTAL_bookings,
       RANK() OVER (ORDER BY COUNT(b.BOOKING_ID) DESC) AS POPUALRITY_RANK
FROM BOOKING b
JOIN BOOKING_DETAIL bd ON b.BOOKING_ID = bd.BOOKING_ID
JOIN FLIGHT f          ON bd.Flight_ID = f.Flight_ID
JOIN ROUTE r           ON f.ROUTE_ID   = r.ROUTE_ID
JOIN AIRPORT a         ON r.ARRIVAL_AIRPORT_ID = a.Airport_ID
GROUP BY a.AIRPORT_NAME, INITCAP(a.AIRPORT_NAME);

```

DESTINATION	TOTAL_BOOKINGS	POPUALRITY_RANK
Kimberley Airport	3	1
O.R. Tambo International	2	2
Polokwane International	1	3
Lanseria International	1	3

Statement 4

This query generates a report showing the route/destination with the highest number of passengers. It uses the COUNT() aggregate function to count the total passengers linked to each route. The query joins the ROUTE, FLIGHT, and BOOKING_DETAIL tables to connect passenger bookings to specific routes. The results are grouped by route code and sorted in descending order so that the route with the highest number of passengers appears first.

```

/*This report shows the destination/route
with the highest number of passengers.*/
SELECT r.ROUTE_CODE,
       COUNT(bd.PASSENGER_ID) AS TOTAL_PASSENGERS
FROM ROUTE r
JOIN FLIGHT f
ON r.ROUTE_ID = f.ROUTE_ID
JOIN BOOKING_DETAIL bd
ON f.FLIGHT_ID = bd.FLIGHT_ID
GROUP BY r.ROUTE_CODE
ORDER BY TOTAL_PASSENGERS DESC;

```

ROUTE_CODE	TOTAL_PASSENGERS
KIM-HLA	2
MQP-PLZ	1
KIM-GRJ	1
CPT-GRJ	1
JNB-CPT	1
JNB-PLZ	1

Statement 5

This query creates a report displaying the routes most frequently used by an aircraft. It uses the COUNT() aggregate function to calculate the total number of flights and passengers for each route. The query joins the ROUTE, FLIGHT, AIRCRAFT, and BOOKING_DETAIL tables, groups the results by route code and aircraft model, and further on sorts them by total passengers in descending order.

```

/*this report shows the the most route that a aircrafts use*/
SELECT r.ROUTE_CODE,
       a.MODEL_ID,
       COUNT(f.FLIGHT_ID) AS TOTAL_FLIGHTS,
       COUNT(bd.PASSENGER_ID) AS TOTAL_PASSENGERS
FROM ROUTE r
JOIN FLIGHT f
ON r.ROUTE_ID = f.ROUTE_ID
JOIN AIRCRAFT a
ON f.AIRCRAFT_ID = a.AIRCRAFT_ID
JOIN BOOKING_DETAIL bd
ON f.FLIGHT_ID = bd.FLIGHT_ID
GROUP BY r.ROUTE_CODE,
         a.MODEL_ID
ORDER BY TOTAL_PASSENGERS DESC;

```

ROUTE_CODE	MODEL_ID	TOTAL_FLIGHTS	TOTAL_PASSENGERS
KIM-HLA	1	2	2
KIM-GRJ	1	1	1
JNB-CPT	3	1	1
MQP-PLZ	2	1	1
JNB-PLZ	1	1	1
CPT-GRJ	1	1	1

Statement 6

1. VW_PASSENGER_LUGGAGE_REPORT — Retrieves all passenger luggage records ordered by weight from heaviest to lightest.
2. VW_STAFF_EXPERIENCE_REPORT — Retrieves all staff records ordered by years of experience from most to least experienced.
3. VW_LATEST_BOOKINGS — Retrieves all booking records ordered by booking date from newest to oldest, showing the most recent bookings first.

4. VW_PASSENGER_ASSISTANCE — Retrieves all passenger assistance records ordered by wheelchair requirement status in descending order.
5. VW_FLIGHT_AVAILABILITY — Retrieves all flight availability records ordered by the number of available seats from highest to lowest.

1	SELECT * FROM VW_PASSENGER_LUGGAGE_REPORT ORDER BY WEIGHT DESC;	1. Passenger luggage report																								
2	SELECT * FROM VW_STAFF_EXPERIENCE_REPORT ORDER BY YEARS_OF_EXPERIENCE DESC;	<table> <tr> <th>⚡ PASSENGER_FNAME</th><th>⚡ PASSENGER_LNAME</th><th>⚡ WEIGHT</th></tr> <tr><td>James</td><td>van der Berg</td><td>30.75</td></tr> <tr><td>Sipho</td><td>Dlamini</td><td>23.5</td></tr> <tr><td>Thabo</td><td>Mokoena</td><td>20</td></tr> <tr><td>Thabo</td><td>Bester</td><td>18.5</td></tr> <tr><td>Thabo</td><td>Bester</td><td>18.5</td></tr> <tr><td>Aisha</td><td>Patel</td><td>8</td></tr> <tr><td>Nomsa</td><td>Khumalo</td><td>7</td></tr> </table>	⚡ PASSENGER_FNAME	⚡ PASSENGER_LNAME	⚡ WEIGHT	James	van der Berg	30.75	Sipho	Dlamini	23.5	Thabo	Mokoena	20	Thabo	Bester	18.5	Thabo	Bester	18.5	Aisha	Patel	8	Nomsa	Khumalo	7
⚡ PASSENGER_FNAME	⚡ PASSENGER_LNAME	⚡ WEIGHT																								
James	van der Berg	30.75																								
Sipho	Dlamini	23.5																								
Thabo	Mokoena	20																								
Thabo	Bester	18.5																								
Thabo	Bester	18.5																								
Aisha	Patel	8																								
Nomsa	Khumalo	7																								
3	/*SHOWS THE NEWEST BOOKING FIRST*/ SELECT * FROM VW_LATEST_BOOKINGS ORDER BY BOOKING_DATE DESC;	2. Staff Experience report																								
4	SELECT * FROM VW_PASSENGER_ASSISTANCE ORDER BY IS_WHEEL_CHAIR DESC;	<table> <tr> <th>⚡ STAFF_FNAME</th><th>⚡ YEARS_OF_EXPERIENCE</th><th>⚡ EXPERIENCE_LEVEL</th></tr> <tr><td>Nomsa</td><td>8</td><td>LESS EXPERIENCED</td></tr> <tr><td>Thandeka</td><td>6</td><td>LESS EXPERIENCED</td></tr> <tr><td>Tholoana</td><td>3</td><td>LESS EXPERIENCED</td></tr> <tr><td>Luyanda</td><td>2</td><td>LESS EXPERIENCED</td></tr> <tr><td>Sandile</td><td>1</td><td>LESS EXPERIENCED</td></tr> </table>	⚡ STAFF_FNAME	⚡ YEARS_OF_EXPERIENCE	⚡ EXPERIENCE_LEVEL	Nomsa	8	LESS EXPERIENCED	Thandeka	6	LESS EXPERIENCED	Tholoana	3	LESS EXPERIENCED	Luyanda	2	LESS EXPERIENCED	Sandile	1	LESS EXPERIENCED						
⚡ STAFF_FNAME	⚡ YEARS_OF_EXPERIENCE	⚡ EXPERIENCE_LEVEL																								
Nomsa	8	LESS EXPERIENCED																								
Thandeka	6	LESS EXPERIENCED																								
Tholoana	3	LESS EXPERIENCED																								
Luyanda	2	LESS EXPERIENCED																								
Sandile	1	LESS EXPERIENCED																								
5	SELECT * FROM VW_FLIGHT_AVAILABILITY ORDER BY AVAILABLE_SEATS DESC;	3. Latest bookings																								
		<table> <tr> <th>⚡ BOOKING_ID</th><th>⚡ BOOKING_DATE</th><th>⚡ BOOKING_STATUS</th></tr> <tr><td>107</td><td>05/DEC/26</td><td>Confirmed</td></tr> <tr><td>106</td><td>05/MAY/26</td><td>Confirmed</td></tr> <tr><td>105</td><td>05/MAY/26</td><td>Confirmed</td></tr> <tr><td>104</td><td>04/MAY/26</td><td>Confirmed</td></tr> <tr><td>103</td><td>03/MAY/26</td><td>Pending</td></tr> <tr><td>102</td><td>02/MAY/26</td><td>Confirmed</td></tr> <tr><td>101</td><td>01/MAY/26</td><td>Confirmed</td></tr> </table>	⚡ BOOKING_ID	⚡ BOOKING_DATE	⚡ BOOKING_STATUS	107	05/DEC/26	Confirmed	106	05/MAY/26	Confirmed	105	05/MAY/26	Confirmed	104	04/MAY/26	Confirmed	103	03/MAY/26	Pending	102	02/MAY/26	Confirmed	101	01/MAY/26	Confirmed
⚡ BOOKING_ID	⚡ BOOKING_DATE	⚡ BOOKING_STATUS																								
107	05/DEC/26	Confirmed																								
106	05/MAY/26	Confirmed																								
105	05/MAY/26	Confirmed																								
104	04/MAY/26	Confirmed																								
103	03/MAY/26	Pending																								
102	02/MAY/26	Confirmed																								
101	01/MAY/26	Confirmed																								
		4. Passenger assistance																								
		<table> <tr> <th>⚡ PASSENGER_FNAME</th><th>⚡ PASSENGER_LNAME</th><th>⚡ IS_WHEEL_CHAIR</th></tr> <tr><td>James</td><td>van der Berg</td><td>Y</td></tr> <tr><td>Thabo</td><td>Mokoena</td><td>Y</td></tr> <tr><td>Thabo</td><td>Bester</td><td>Y</td></tr> <tr><td>Aisha</td><td>Patel</td><td>N</td></tr> <tr><td>Nomsa</td><td>Khumalo</td><td>N</td></tr> <tr><td>Sipho</td><td>Dlamini</td><td>N</td></tr> </table>	⚡ PASSENGER_FNAME	⚡ PASSENGER_LNAME	⚡ IS_WHEEL_CHAIR	James	van der Berg	Y	Thabo	Mokoena	Y	Thabo	Bester	Y	Aisha	Patel	N	Nomsa	Khumalo	N	Sipho	Dlamini	N			
⚡ PASSENGER_FNAME	⚡ PASSENGER_LNAME	⚡ IS_WHEEL_CHAIR																								
James	van der Berg	Y																								
Thabo	Mokoena	Y																								
Thabo	Bester	Y																								
Aisha	Patel	N																								
Nomsa	Khumalo	N																								
Sipho	Dlamini	N																								
		5. Flight availability																								

FLIGHT_ID	FLIGHT_STATUS	AVAILABLE_SEATS
137	Scheduled	160
117	Scheduled	160
127	Scheduled	160
132	Scheduled	155
122	Scheduled	155
129	Cancelled	150
119	Scheduled	150
375	Cancelled	150
90	Scheduled	150
145	Cancelled	150
144	Scheduled	150
139	Cancelled	150
143	Scheduled	148

Statement 7

This statement creates a view called VW_PASSENGER_LUGGAGE_REPORT that links each passenger to their associated luggage weight. It joins the PASSENGER, BOOKING_DETAIL, and LUGGAGE tables to retrieve each passenger's first and last name alongside their luggage weight. The following SELECT then queries this view and returns all records ordered by weight in descending order, showing the heaviest luggage first.

```
CREATE OR REPLACE VIEW VW_PASSENGER_LUGGAGE_REPORT AS
SELECT p.PASSENGER_FNAME,
       p.PASSENGER_LNAME,
       l.WEIGHT
FROM PASSENGER p
JOIN BOOKING_DETAIL bd
  ON p.PASSENGER_ID = bd.PASSENGER_ID
JOIN LUGGAGE l
  ON bd.BOOKING_ID = l.BOOKING_DETAIL_ID;
```

PASSENGER_FNAME	PASSENGER_LNAME	WEIGHT
James	van der Berg	30.75
Sipho	Dlamini	23.5
Thabo	Mokoena	20
Thabo	Bester	18.5
Thabo	Bester	18.5
Aisha	Patel	8
Nomsa	Khumalo	7

7 rows selected.

Statement 8

This statement calculates the average luggage weight for each aircraft model. It joins the LUGGAGE, FLIGHT, AIRCRAFT, and MODEL tables together, then groups the results

by aircraft ID and model name to produce the average luggage weight carried per aircraft model.

```
/*HEAVIEST LUGGAGE FLIGHTS (HAVING)*/
SELECT
    ROUTE_ID,
    SUM(TOTAL_LUGGAGE_WEIGHT) AS TOTAL_LOAD
FROM V_AIRCRAFT_ROUTE_REPORT
GROUP BY ROUTE_ID
HAVING SUM(TOTAL_LUGGAGE_WEIGHT) > 100;
```

ROUTE_ID	TOTAL_LOAD
45	38.5
19	30.75

Statement 9

This statement identifies routes where the total luggage weight exceeds 100 units. It groups records from the V_AIRCRAFT_ROUTE_REPORT view by route and uses the HAVING clause to filter out only those routes whose combined luggage weight surpasses the threshold, returning the route ID and its total load.

```
/*Average luggage weight*/
SELECT a.AIRCRAFT_ID,
       m.MODEL_NAME,
       AVG(l.Weight) AS Average_Luggage_Weight
FROM LUGGAGE l
JOIN FLIGHT f
ON l.FLIGHT_ID = f.FLIGHT_ID
JOIN AIRCRAFT a
ON f.AIRCRAFT_ID = a.AIRCRAFT_ID
JOIN MODEL m
ON a.MODEL_ID = m.MODEL_ID
GROUP BY a.AIRCRAFT_ID, m.MODEL_NAME;
```

AIRCRAFT_ID	MODEL_NAME	AVERAGE_LUGGAGE_WEIGHT
5	Airbus A320	18.5

Statement 10

This statement displays the number of bookings made per day. It uses TRUNC to strip the time portion from the booking date, groups all records by day, and counts the total bookings for each day from the BOOKING table.

```
/*display number of booking per day*/
SELECT TRUNC(BOOKING_DATE, 'DD') AS booking_day,
       COUNT(*) AS daily_bookings
FROM BOOKING
GROUP BY TRUNC(BOOKING_DATE, 'DD');
```

```
BOOKING_D DAILY_BOOKINGS
```

```
-----
03/MAY/26      1
05/MAY/26      2
04/MAY/26      1
02/MAY/26      1
05/DEC/26      1
01/MAY/26      1
```

```
6 rows selected.
```

Statement 11

This statement retrieves all luggage weights from the LUGGAGE table and truncates each weight value to one decimal place using the TRUNC function. This is used to standardize weight values for display or reporting purposes without rounding.

```
/*round off weight to 2 decimals*/
SELECT WEIGHT,
       TRUNC(WEIGHT, 1) AS truncated_weight
FROM LUGGAGE;
```

```
WEIGHT TRUNCATED_WEIGHT
-----
23.5      23.5
7         7
30.75     30.7
8         8
20        20
18.5      18.5
18.5      18.5
```

```
7 rows selected.
```

Statement 12

This statement retrieves all months where the total number of bookings exceeds the overall monthly average. It uses a subquery to calculate the average total bookings across all months from the V_MONTHLY_BOOKING_STATS view, then filters and returns only the months that performed above that average.

```
-- HAVING: MONTHS ABOVE AVERAGE BOOKINGS
SELECT BOOKING_MONTH,
       TOTAL_BOOKINGS
FROM V_MONTHLY_BOOKING_STATS
WHERE TOTAL_BOOKINGS >
(
  SELECT AVG(TOTAL_BOOKINGS)
  FROM V_MONTHLY_BOOKING_STATS
);
```



```
BOOKING TOTAL_BOOKINGS
-----
2026-05                6
```

Statement 13

This statement retrieves the full record of the month that had the highest number of bookings. It queries the V_MONTHLY_BOOKING_STATS view and uses a subquery to first determine the maximum value of TOTAL_BOOKINGS across all months. The outer query then returns only the month whose total bookings match that maximum value, effectively identifying the single busiest booking month on record.

```
-- BUSIEST MONTH ONLY (SUBQUERY)
SELECT *
FROM V_MONTHLY_BOOKING_STATS
WHERE TOTAL_BOOKINGS =
(
    SELECT MAX(TOTAL_BOOKINGS)
    FROM V_MONTHLY_BOOKING_STATS
);
```

```
BOOKING TOTAL_BOOKINGS CONFIRMED_BOOKINGS CANCELLED_BOOKINGS
-----
2026-05                6                0                0
```

Statement 14

This statement retrieves the full record of the flight or route that carried the highest number of passengers. It queries the V_AIRCRAFT_ROUTE_REPORT view and uses a subquery to find the maximum value of TOTAL_PASSENGERS across all records. The outer query then returns only the entry whose passenger count matches that maximum, identifying the single busiest flight or route in terms of passenger volume.

```
/*MOST BUSY FLIGHTS (SUBQUERY)*/
SELECT *
FROM V_AIRCRAFT_ROUTE_REPORT
WHERE TOTAL_PASSENGERS = (
    SELECT MAX(TOTAL_PASSENGERS)
    FROM V_AIRCRAFT_ROUTE_REPORT
);
```

```
FLIGHT_ID  ROUTE_ID TOTAL_PASSENGERS TOTAL_LUGGAGE_WEIGHT
-----
55         45                2                38.5
```

Statement 15

This query we used the TO_CHAR date function to format the Duty Start Time into a more readable format, displaying the day, month, year and time. We also used ROWNUM to limit the results to only the first 5 rows, giving us a clean and concise view of the data, showing the Flight ID, Staff ID, Start Time and Assignment Status."

```

SELECT FLIGHT_ID,
       STAFF_ID,
       TO_CHAR(DUTY_START_TIME, 'DD/MON/YY HH24:MI') AS START_TIME,
       ASSIGNMENT_STATUS
FROM FLIGHT_ASSIGNMENT
WHERE ROWNUM <= 5
ORDER BY DUTY_START_TIME;

```

FLIGHT_ID	STAFF_ID	START_TIME	ASSIGNMENT_STATUS
40	1	04/JUN/26 06:00	Completed
40	2	04/JUN/26 06:00	Completed
55	4	05/JUN/26 06:00	Completed
55	7	05/JUN/26 06:00	Completed
117	5	26/JUN/26 04:45	Completed

Extra Functionality**Statement 16****Trigger**

This trigger fires automatically after a new flight record is inserted into the FLIGHT table. For each new flight added, it updates the ROUTE table by incrementing the frequency count of the route associated with that flight. This ensures that the ROUTE table always reflects an accurate and up-to-date count of how many flights have been scheduled on each route, without requiring manual updates from the application.

```

/*trigger for frequency*/
CREATE OR REPLACE TRIGGER trg_update_route_frequency
AFTER INSERT ON FLIGHT
FOR EACH ROW
BEGIN
    UPDATE ROUTE
    SET FREQUENCY = FREQUENCY + 1
    WHERE ROUTE_ID = :NEW.ROUTE_ID;
END;

```

Statement 17**CASE**

The CASE expression is used to conditionally count bookings by status. For each booking record, it evaluates whether the booking status matches a specific value and returns 1 if true or 0 if false. When combined with SUM, this effectively counts only the records that meet each condition, allowing confirmed and cancelled bookings to be tallied separately within the same query without needing multiple subqueries or joins.

```

/*MONTHLY BOOKING REPORT*/
CREATE OR REPLACE VIEW V_MONTHLY_BOOKING_STATS AS
SELECT TO_CHAR(BOOKING_DATE, 'YYYY-MM') AS BOOKING_MONTH,
       COUNT(BOOKING_ID) AS TOTAL_BOOKINGS,
       SUM(CASE WHEN BOOKING_STATUS = 'CONFIRMED' THEN 1 ELSE 0 END) AS CONFIRMED_BOOKINGS,
       SUM(CASE WHEN BOOKING_STATUS = 'CANCELLED' THEN 1 ELSE 0 END) AS CANCELLED_BOOKINGS
FROM BOOKING
GROUP BY TO_CHAR(BOOKING_DATE, 'YYYY-MM');

```

BOOKING	TOTAL_BOOKINGS	CONFIRMED_BOOKINGS	CANCELLED_BOOKINGS
2026-12	1	0	0
2026-05	6	0	0